creating a Pandas Series:

In [1]:
```
import pandas as pd
import numpy as np

# 1. Using List
list_data = [10, 20, 30, 40]
series_list = pd.Series(list_data)
print("Series from List:\n", series_list, "\n")

# 2. List with Custom Index
list_custom = [100, 200, 300]
series_custom = pd.Series(list_custom, index=['a', 'b', 'c'])
print("Series from List with Custom Index:\n", series_custom, "\n")

# 3. Using Tuple
tuple_data = (5, 10, 15)
series_tuple = pd.Series(tuple_data)
print("Series from Tuple:\n", series_tuple, "\n")

# 4. Using Dictionary
dict_data = {'A': 1000, 'B': 2000, 'C': 3000}
series_dict = pd.Series(dict_data)
print("Series from Dictionary:\n", series_dict, "\n")

# 5. Using NumPy Array
numpy_data = np.array([1, 2, 3, 4])
series_numpy = pd.Series(numpy_data)
print("Series from NumPy Array:\n", series_numpy, "\n")

# 6. Using Range
range_data = range(1, 6)
series_range = pd.Series(range_data)
print("Series from Range:\n", series_range, "\n")
```

Vishal Acharya

```
Series from List:
 0    10
1    20
2    30
3    40
dtype: int64

Series from List with Custom Index:
 a    100
b    200
c    300
dtype: int64

Series from Tuple:
 0     5
1    10
2    15
dtype: int64

Series from Dictionary:
 A    1000
B    2000
C    3000
dtype: int64

Series from NumPy Array:
 0    1
1    2
2    3
3    4
dtype: int32

Series from Range:
 0    1
1    2
2    3
3    4
4    5
dtype: int64
```

Vishal Acharya

creating a Pandas DataFrame:

In [4]:
```python
import pandas as pd
import numpy as np

# 1. Dictionary of Lists
data_dict_list = {
    'Name': ['A', 'B', 'C'],
    'Age': [20, 21, 22]
}
df1 = pd.DataFrame(data_dict_list)
print("1. DataFrame from Dictionary of Lists:\n", df1, "\n")

# 2. List of Lists
data_list_list = [
    ['A', 20],
    ['B', 21],
```

```
    ['C', 22]
]
df2 = pd.DataFrame(data_list_list, columns=['Name', 'Age'])
print("2. DataFrame from List of Lists:\n", df2, "\n")

# 3. List of Dictionaries
data_list_dict = [
    {'Name': 'A', 'Age': 20},
    {'Name': 'B', 'Age': 21},
    {'Name': 'C', 'Age': 22}
]
df3 = pd.DataFrame(data_list_dict)
print("3. DataFrame from List of Dictionaries:\n", df3, "\n")

# 4. Dictionary of Series
data_dict_series = {
    'Name': pd.Series(['A', 'B', 'C']),
    'Age': pd.Series([20, 21, 22])
}
df4 = pd.DataFrame(data_dict_series)
print("4. DataFrame from Dictionary of Series:\n", df4, "\n")

# 5. NumPy Array
array_data = np.array([
    ['A', 20],
    ['B', 21],
    ['C', 22]
])
df5 = pd.DataFrame(array_data, columns=['Name', 'Age'])
print("5. DataFrame from NumPy Array:\n", df5, "\n")

# 6. CSV File (Example)
# Make sure 'sample.csv' exists in your working directory

df6 = pd.read_csv("sample.csv")
print("6. DataFrame from CSV:\n", df6, "\n")


# 7. Empty DataFrame
df7 = pd.DataFrame()
print("7. Empty DataFrame:\n", df7)
```

Vishal Acharya

```
1. DataFrame from Dictionary of Lists:
   Name  Age
0    A   20
1    B   21
2    C   22

2. DataFrame from List of Lists:
   Name  Age
0    A   20
1    B   21
2    C   22

3. DataFrame from List of Dictionaries:
   Name  Age
0    A   20
1    B   21
2    C   22

4. DataFrame from Dictionary of Series:
   Name  Age
0    A   20
1    B   21
2    C   22

5. DataFrame from NumPy Array:
   Name Age
0    A  20
1    B  21
2    C  22

6. DataFrame from CSV:
    A  B  C  D
0  1  2  3  4
1  5  4  7  8

7. Empty DataFrame:
 Empty DataFrame
Columns: []
Index: []
```

set_index(keys, drop=True, inplace=True)

keys → Column name(s) to be set as the new index

drop=True → Removes the column(s) used as index from the DataFrame

inplace=True → Modifies the original DataFrame instead of creating a new one

reset_index()

reset_index() → Resets the index to default integer index and converts old index into a column

In [6]:
```python
import pandas as pd

# Create DataFrame
data = {
```

Vishal Acharya

```python
    'ID': [101, 102, 103],
    'Name': ['A', 'B', 'C'],
    'Marks': [85, 90, 95]
}

df = pd.DataFrame(data)
print("Original DataFrame:\n", df, "\n")

# 1. set_index() with drop=True (default)
df1 = df.set_index('ID')
print("After set_index('ID') [drop=True]:\n", df1, "\n")

# 2. set_index() with drop=False
df2 = df.set_index('ID', drop=False)
print("After set_index('ID', drop=False):\n", df2, "\n")

# 3. set_index() with inplace=True
df_copy = df.copy()
df_copy.set_index('ID', inplace=True)
print("After set_index(inplace=True):\n", df_copy, "\n")

# 4. reset_index()
df_reset = df1.reset_index()
print("After reset_index():\n", df_reset)
```

```
Original DataFrame:
     ID Name  Marks
0  101    A     85
1  102    B     90
2  103    C     95

After set_index('ID') [drop=True]:
     Name  Marks
ID
101    A     85
102    B     90
103    C     95

After set_index('ID', drop=False):
       ID Name  Marks
ID
101  101    A     85
102  102    B     90
103  103    C     95

After set_index(inplace=True):
     Name  Marks
ID
101    A     85
102    B     90
103    C     95

After reset_index():
     ID Name  Marks
0  101    A     85
1  102    B     90
2  103    C     95
```

DataFrame Attributes

Vishal Acharya

shape → Returns tuple (rows, columns) of the DataFrame

size → Total number of elements (rows × columns)

ndim → Number of dimensions (always 2 for DataFrame)

columns → Returns column labels (column names)

index → Returns row labels (index values)

dtypes → Shows data type of each column

values → Returns data as a NumPy array

Vishal Acharya

In [7]:
```
import pandas as pd

df = pd.DataFrame({
    'A': [1, 2],
    'B': [3, 4]
})

print(df.shape)    # (2, 2)
print(df.size)     # 4
print(df.ndim)     # 2
print(df.columns)  # Index(['A', 'B'])
print(df.index)    # RangeIndex(0, 2)
print(df.dtypes)   # int64 for both columns
print(df.values)   # [[1 3], [2 4]]
```

```
(2, 2)
4
2
Index(['A', 'B'], dtype='object')
RangeIndex(start=0, stop=2, step=1)
A    int64
B    int64
dtype: object
[[1 3]
 [2 4]]
```

loc → Access data using row/column labels (inclusive slicing)

iloc → Access data using row/column positions (exclusive slicing)

In [9]:
```
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'Name': ['A', 'B', 'C'],
    'Marks': [85, 90, 95]
}, index=[101, 102, 103])

print("Original DataFrame:\n", df, "\n")
```

```python
# 🔹 loc → Label-based indexing (uses index names)
print("loc single row:\n", df.loc[101], "\n")   # Access row by label

# 🔹 loc slicing (inclusive)
print("loc slicing:\n", df.loc[101:102], "\n")

# 🔹 loc edit value
df.loc[101, 'Marks'] = 88
print("After loc edit:\n", df, "\n")

# 🔹 loc add new column
df.loc[:, 'Grade'] = ['B', 'A', 'A']
print("After adding column using loc:\n", df, "\n")

# 🔹 loc add new row
df.loc[104] = ['D', 80, 'C']
print("After adding row using loc:\n", df, "\n")


# 🔹 iloc → Position-based indexing (uses integer position)
print("iloc single row:\n", df.iloc[0], "\n")   # First row

# 🔹 iloc slicing (exclusive end)
print("iloc slicing:\n", df.iloc[0:2], "\n")

# 🔹 iloc edit value
df.iloc[0, 1] = 89
print("After iloc edit:\n", df, "\n")
```

Vishal Acharya

```
Original DataFrame:
     Name  Marks
101    A     85
102    B     90
103    C     95

loc single row:
 Name      A
Marks    85
Name: 101, dtype: object

loc slicing:
     Name  Marks
101    A     85
102    B     90

After loc edit:
     Name  Marks
101    A     88
102    B     90
103    C     95

After adding column using loc:
     Name  Marks Grade
101    A     88     B
102    B     90     A
103    C     95     A

After adding row using loc:
     Name  Marks Grade
101    A     88     B
102    B     90     A
103    C     95     A
104    D     80     C

iloc single row:
 Name      A
Marks    88
Grade     B
Name: 101, dtype: object

iloc slicing:
     Name  Marks Grade
101    A     88     B
102    B     90     A

After iloc edit:
     Name  Marks Grade
101    A     89     B
102    B     90     A
103    C     95     A
104    D     80     C
```

& (AND) → Both conditions must be True

| (OR) → At least one condition must be True

~ (NOT) → Reverse the condition

Vishal Acharya

In [16]:
```
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'Name': ['A', 'B', 'C', 'D'],
    'Marks': [85, 90, 75, 95],
    'Age': [20, 21, 19, 22],
    'Dept': ['IT', 'HR', 'IT', 'Finance']
})

print("DataFrame:\n", df, "\n")

#  🔹 AND (&) → Marks > 80 AND Age > 20
print("AND condition:\n",
      df[(df['Marks'] > 80) & (df['Age'] > 20)], "\n")

#  🔹 OR (|) → Marks > 90 OR Age < 20
print("OR condition:\n",
      df[(df['Marks'] > 90) | (df['Age'] < 20)], "\n")

#  🔹 NOT (~) → NOT Dept == 'IT'
print("NOT condition:\n",
      df[~(df['Dept'] == 'IT')], "\n")

#  🔹 Combined conditions
print("Combined (& and |):\n",
      df[(df['Dept'] == 'IT') & (df['Marks'] > 80)], "\n")
```

Vishal Acharya

```
DataFrame:
   Name  Marks  Age     Dept
0    A     85   20       IT
1    B     90   21       HR
2    C     75   19       IT
3    D     95   22  Finance

AND condition:
   Name  Marks  Age     Dept
1    B     90   21       HR
3    D     95   22  Finance

OR condition:
   Name  Marks  Age     Dept
2    C     75   19       IT
3    D     95   22  Finance

NOT condition:
   Name  Marks  Age     Dept
1    B     90   21       HR
3    D     95   22  Finance

Combined (& and |):
   Name  Marks  Age Dept
0    A     85   20   IT

NOT IN condition:
   Name  Marks  Age     Dept
3    D     95   22  Finance

Between Marks 80 and 95:
   Name  Marks  Age     Dept
0    A     85   20       IT
1    B     90   21       HR
3    D     95   22  Finance

Name starts with 'A':
   Name  Marks  Age Dept
0    A     85   20   IT
```

In [17]:

```python
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'Name': ['A', 'B', 'C', 'D'],
    'Marks': [85, 90, 75, 95],
    'Age': [20, 21, 19, 22],
    'Dept': ['IT', 'HR', 'IT', 'Finance']
})

print("DataFrame:\n", df, "\n")

# Combined condition:
# (Marks > 80 AND Age > 20) OR NOT Dept == 'IT'

result = df[((df['Marks'] > 80) & (df['Age'] > 20)) | ~(df['Dept'] == 'IT')]

print("Filtered Result:\n", result)
```

Vishal Acharya

```
DataFrame:
   Name  Marks  Age     Dept
0    A     85   20       IT
1    B     90   21       HR
2    C     75   19       IT
3    D     95   22  Finance

Filtered Result:
   Name  Marks  Age     Dept
1    B     90   21       HR
3    D     95   22  Finance
```

df.head(n) → Shows first n rows (n=5 default)

df.tail(n) → Shows last n rows (n=5 default)

df.sample(n) → Shows n random rows (n=1 default)

In [10]:
```python
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'Name': ['A', 'B', 'C', 'D', 'E', 'F'],
    'Marks': [85, 90, 95, 88, 76, 92]
})
# head() → Returns first n rows (default 5)
print("head():\n", df.head(3), "\n")
# tail() → Returns last n rows (default 5)
print("tail():\n", df.tail(3), "\n")
# sample() → Returns random n rows from DataFrame
print("sample():\n", df.sample(2))
```

```
head():
   Name  Marks
0    A     85
1    B     90
2    C     95

tail():
   Name  Marks
3    D     88
4    E     76
5    F     92

sample():
   Name  Marks
5    F     92
0    A     85
```

df.info() → Gives complete summary of DataFrame (columns, dtypes, non-null values, memory usage)

In [11]:
```python
import pandas as pd
# Create DataFrame
df = pd.DataFrame({
    'Name': ['A', 'B', 'C'],
    'Marks': [85, 90, 95]})
# info() → Displays summary (columns, data types, non-null count, memory usage)
df.info()
```

Vishal Acharya

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    3 non-null      object
 1   Marks   3 non-null      int64
dtypes: int64(1), object(1)
memory usage: 176.0+ bytes
```

df.describe(include=None, exclude=None, percentiles=None)

include → Specifies which data types to include (e.g., 'all', ['int', 'object'])

exclude → Specifies which data types to exclude from summary

percentiles → Defines custom percentile values to display (e.g., [0.25, 0.5, 0.75])

In [12]:

```python
import pandas as pd

df = pd.DataFrame({
    'A': [10, 20, 30],
    'B': [1.5, 2.5, 3.5],
    'C': ['X', 'Y', 'Z']
})

# include → include all columns
print(df.describe(include='all'), "\n")

# exclude → exclude object type
print(df.describe(exclude=['object']), "\n")

# percentiles → custom percentiles
print(df.describe(percentiles=[0.2, 0.5, 0.8]))
```

Vishal Acharya

```
           A    B    C
count    3.0  3.0    3
unique   NaN  NaN    3
top      NaN  NaN    X
freq     NaN  NaN    1
mean    20.0  2.5  NaN
std     10.0  1.0  NaN
min     10.0  1.5  NaN
25%     15.0  2.0  NaN
50%     20.0  2.5  NaN
75%     25.0  3.0  NaN
max     30.0  3.5  NaN

          A    B
count   3.0  3.0
mean   20.0  2.5
std    10.0  1.0
min    10.0  1.5
25%    15.0  2.0
50%    20.0  2.5
75%    25.0  3.0
max    30.0  3.5

          A    B
count   3.0  3.0
mean   20.0  2.5
std    10.0  1.0
min    10.0  1.5
20%    14.0  1.9
50%    20.0  2.5
80%    26.0  3.1
max    30.0  3.5
```

Frequency / Unique (One-Line)

unique() → Returns all distinct values from a column

nunique() → Returns the count of unique values

df.value_counts(ascending=True) → Returns frequency of unique rows/values sorted in ascending order

In [13]:

```python
import pandas as pd

df = pd.DataFrame({
    'Dept': ['IT', 'HR', 'IT', 'Finance', 'HR', 'IT']
})

# unique() → distinct values
print("unique():", df['Dept'].unique(), "\n")

# nunique() → count of unique values
print("nunique():", df['Dept'].nunique(), "\n")

# value_counts() → frequency count (ascending order)
print("value_counts():\n", df['Dept'].value_counts(ascending=True))
```

Vishal Acharya

```
unique(): ['IT' 'HR' 'Finance']

nunique(): 3

value_counts():
 Finance    1
HR         2
IT         3
Name: Dept, dtype: int64
```

Missing Value Functions

df.isnull() → Returns True/False for missing (NaN) values→ detect missing

df.isnull().sum() → Returns the total number of missing (NaN) values in each column

df.fillna(value, inplace=False) → Replaces missing values with given value → replace missing

df.dropna(axis=0, how='any', thresh=None, subset=None, inplace=False) → Removes rows/columns with missing values based on conditions → remove missing

1. fillna()

value → Value used to replace NaN

inplace → Modify original DataFrame if True

2. dropna()

axis → 0 = rows, 1 = columns

how → 'any' = drop if any NaN, 'all' = drop if all NaN

thresh → Minimum non-NaN values required to keep row/column

subset → Specific columns to check

inplace → Modify original DataFrame if True

In [16]:

```python
import pandas as pd
import numpy as np

# Create DataFrame with missing values
df = pd.DataFrame({
    'Name': ['A', 'B', 'C', 'D'],
    'Marks': [85, np.nan, 95, np.nan],
    'Dept': ['IT', 'HR', None, 'Finance']
})

print("Original DataFrame:\n", df, "\n")

# isnull() → check missing values
print("isnull():\n", df.isnull(), "\n")
print("isnull().sum()\n")
print(df.isnull().sum())
```

Vishal Acharya

```python
# fillna() → replace missing values
df_filled = df.fillna({'Marks': 0, 'Dept': 'Unknown'})
print("After fillna():\n", df_filled, "\n")

# dropna() → drop rows with any missing values
df_dropped = df.dropna()
print("After dropna():\n", df_dropped, "\n")

# dropna() with subset → check only specific column
df_subset = df.dropna(subset=['Marks'])
print("After dropna(subset=['Marks']):\n", df_subset)
```

```
Original DataFrame:
   Name  Marks     Dept
0    A   85.0       IT
1    B    NaN       HR
2    C   95.0     None
3    D    NaN  Finance

isnull():
     Name  Marks   Dept
0  False  False  False
1  False   True  False
2  False  False   True
3  False   True  False

isnull().sum()

Name     0
Marks    2
Dept     1
dtype: int64
After fillna():
   Name  Marks     Dept
0    A   85.0       IT
1    B    0.0       HR
2    C   95.0  Unknown
3    D    0.0  Finance

After dropna():
   Name  Marks Dept
0    A   85.0   IT

After dropna(subset=['Marks']):
   Name  Marks  Dept
0    A   85.0    IT
2    C   95.0  None
```

df.drop(labels=None, axis=0, index=None, columns=None, inplace=False)

labels → Name(s) of row(s) or column(s) to remove

axis → 0 = drop rows, 1 = drop columns

index → Specify row labels to drop (alternative to labels)

columns → Specify column names to drop

Vishal Acharya

inplace → If True, modifies original DataFrame

In [19]:
```
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'Name': ['A', 'B', 'C'],
    'Marks': [85, 90, 95],
    'Dept': ['IT', 'HR', 'Finance']
}, index=[101, 102, 103])

print("Original DataFrame:\n", df, "\n")

# Drop row using labels (axis=0)
print("Drop row (labels):\n", df.drop(labels=101, axis=0), "\n")

# Drop row using index parameter
print("Drop row (index):\n", df.drop(index=102), "\n")

# Drop column using labels (axis=1)
print("Drop column (labels):\n", df.drop(labels='Dept', axis=1), "\n")

# Drop column using columns parameter
print("Drop column (columns):\n", df.drop(columns=['Marks']), "\n")

# Drop with inplace=True
df_copy = df.copy()
df_copy.drop(index=103, inplace=True)
print("After inplace drop:\n", df_copy)

#another option for inplace
print("for permanent option LHS=RHS")
df=df.drop(index=103)
```

Vishal Acharya

```
Original DataFrame:
     Name  Marks     Dept
101    A     85       IT
102    B     90       HR
103    C     95  Finance

Drop row (labels):
     Name  Marks     Dept
102    B     90       HR
103    C     95  Finance

Drop row (index):
     Name  Marks     Dept
101    A     85       IT
103    C     95  Finance

Drop column (labels):
     Name  Marks
101    A     85
102    B     90
103    C     95

Drop column (columns):
     Name     Dept
101    A       IT
102    B       HR
103    C  Finance

After inplace drop:
     Name  Marks Dept
101    A     85   IT
102    B     90   HR
for permanent option LHS=RHS
```

df.duplicated(subset=None, keep='first') → Returns True for duplicate rows (keeps first occurrence as False)

df.drop_duplicates(subset=None, keep='first', inplace=False) → Removes duplicate rows and keeps first occurrence

subset → Column(s) to check for duplicates

keep → 'first', 'last', or False (mark/remove all duplicates)

inplace → Modify original DataFrame if True

In [23]:
```python
import pandas as pd

# Create DataFrame with duplicates
df = pd.DataFrame({
    'Name': ['A', 'B', 'A', 'C', 'B'],
    'Marks': [85, 90, 85, 95, 90]
})

print("Original DataFrame:\n", df, "\n")

# duplicated() → find duplicates
print("duplicated():\n", df.duplicated(), "\n")
```

Vishal Acharya

```python
#duplicated().sum
print("duplicated().sum\n",df.duplicated().sum,"\n")
# duplicated() with subset
print("duplicated(subset=['Name']):\n", df.duplicated(subset=['Name']), "\n")

# drop_duplicates() → remove duplicates
df_unique = df.drop_duplicates()
print("After drop_duplicates():\n", df_unique, "\n")

# drop_duplicates() keep='last'
df_last = df.drop_duplicates(keep='last')
print("After drop_duplicates(keep='last'):\n", df_last)
```

```
Original DataFrame:
   Name  Marks
0     A     85
1     B     90
2     A     85
3     C     95
4     B     90

duplicated():
 0    False
1    False
2     True
3    False
4     True
dtype: bool

duplicated().sum
 <bound method NDFrame._add_numeric_operations.<locals>.sum of 0    False
1    False
2     True
3    False
4     True
dtype: bool>

duplicated(subset=['Name']):
 0    False
1    False
2     True
3    False
4     True
dtype: bool

After drop_duplicates():
   Name  Marks
0     A     85
1     B     90
3     C     95

After drop_duplicates(keep='last'):
   Name  Marks
2     A     85
3     C     95
4     B     90
```

IQR method → Detects outliers using Q1, Q3 and removes values outside (Q1−1.5×IQR, Q3+1.5×IQR)

Vishal Acharya

Vishal Acharya

In [18]:
```python
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'Marks': [50, 55, 60, 65, 70, 75, 200]  # 200 is an outlier
})

print("Original Data:\n", df, "\n")

# Step 1: Calculate Q1 and Q3
Q1 = df['Marks'].quantile(0.25)
Q3 = df['Marks'].quantile(0.75)

# Step 2: Calculate IQR
IQR = Q3 - Q1

# Step 3: Define bounds
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

print("Lower Bound:", lower)
print("Upper Bound:", upper, "\n")

# Step 4: Detect outliers
outliers = df[(df['Marks'] < lower) | (df['Marks'] > upper)]
print("Outliers:\n", outliers, "\n")

# Step 5: Remove outliers
df_clean = df[(df['Marks'] >= lower) & (df['Marks'] <= upper)]
print("Data after removing outliers:\n", df_clean)
```

```
Original Data:
    Marks
0     50
1     55
2     60
3     65
4     70
5     75
6    200

Lower Bound: 35.0
Upper Bound: 95.0

Outliers:
    Marks
6    200

Data after removing outliers:
    Marks
0     50
1     55
2     60
3     65
4     70
5     75
```

Sorting

df.sort_values(by, axis=0, ascending=True, inplace=False) → Sorts DataFrame by one or more column values

df.sort_index(axis=0, ascending=True, inplace=False) → Sorts DataFrame by index labels

by → Column name(s) (list for multiple columns)

axis → 0 = rows, 1 = columns

ascending → True = ascending, False = descending (can pass list)

inplace → Modify original DataFrame

In [24]:
```
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'Name': ['A', 'B', 'C', 'D'],
    'Dept': ['IT', 'HR', 'IT', 'HR'],
    'Marks': [85, 90, 85, 80]
}, index=[3, 1, 4, 2])

print("Original DataFrame:\n", df, "\n")

#  Sort by one column
print("Sort by Marks:\n", df.sort_values(by='Marks'), "\n")

#  Sort by multiple columns (Dept then Marks)
print("Sort by Dept & Marks:\n",
      df.sort_values(by=['Dept', 'Marks']), "\n")

#  Different order (Dept ascending, Marks descending)
print("Multi-sort with different order:\n",
      df.sort_values(by=['Dept', 'Marks'], ascending=[True, False]), "\n")

#  Sort by index
print("Sort by index:\n", df.sort_index(), "\n")

#  Sort by index column
print("Sort by index column:\n", df.sort_index(axis=1), "\n")

#  Sort index descending
print("Sort index descending:\n", df.sort_index(ascending=False))
```

Vishal Acharya

```
Original DataFrame:
   Name Dept  Marks
3    A   IT     85
1    B   HR     90
4    C   IT     85
2    D   HR     80

Sort by Marks:
   Name Dept  Marks
2    D   HR     80
3    A   IT     85
4    C   IT     85
1    B   HR     90

Sort by Dept & Marks:
   Name Dept  Marks
2    D   HR     80
1    B   HR     90
3    A   IT     85
4    C   IT     85

Multi-sort with different order:
   Name Dept  Marks
1    B   HR     90
2    D   HR     80
3    A   IT     85
4    C   IT     85

Sort by index:
   Name Dept  Marks
1    B   HR     90
2    D   HR     80
3    A   IT     85
4    C   IT     85

Sort by index column:
   Dept  Marks Name
3   IT     85    A
1   HR     90    B
4   IT     85    C
2   HR     80    D

Sort index descending:
   Name Dept  Marks
4    C   IT     85
3    A   IT     85
2    D   HR     80
1    B   HR     90
```

Statistics Functions

sum(axis=0, numeric_only=True) → Returns column-wise total of numeric values only

mean(axis=0, numeric_only=True) → Returns column-wise average of numeric values only

max(axis=0, numeric_only=True) → Returns maximum value in each numeric column

Vishal Acharya

min(axis=0, numeric_only=True) → Returns minimum value in each numeric column (you wrote mod, correct is min)

df.mode(axis=0/1, numeric_only=True) → Returns the most frequent value(s) column-wise or row-wise

median(axis=0, numeric_only=True) → Returns median of numeric columns

count() → Counts non-null values in each column

quantile(q, numeric_only=True) → Returns specified percentile (e.g., 0.25, 0.5, 0.75)

numeric_only=True → Ignores non-numeric columns (like strings)

axis=0 → Operates column-wise (down rows)

axis=1 → Operates row-wise (across columns)

In [3]:
```
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'Name': ['A', 'B', 'C'],
    'Marks': [85, 90, 90],
    'Age': [20, 21, 21]
})

print("DataFrame:\n", df, "\n")

#  COLUMN-WISE (axis=0)

print("sum(axis=0):\n", df.sum(axis=0, numeric_only=True), "\n")
print("mean(axis=0):\n", df.mean(axis=0, numeric_only=True), "\n")
print("max(axis=0):\n", df.max(axis=0, numeric_only=True), "\n")
print("min(axis=0):\n", df.min(axis=0, numeric_only=True), "\n")
print("median(axis=0):\n", df.median(axis=0, numeric_only=True), "\n")

#  mode (most frequent value)
print("mode(axis=0):\n", df.mode(numeric_only=True), "\n")

#  ROW-WISE (axis=1)

print("sum(axis=1):\n", df.sum(axis=1, numeric_only=True), "\n")
print("mean(axis=1):\n", df.mean(axis=1, numeric_only=True), "\n")
print("max(axis=1):\n", df.max(axis=1, numeric_only=True), "\n")
print("min(axis=1):\n", df.min(axis=1, numeric_only=True), "\n")
print("median(axis=1):\n", df.median(axis=1, numeric_only=True), "\n")

#  mode row-wise
print("mode(axis=1):\n", df.mode(axis=1, numeric_only=True), "\n")

#  count
print("count(axis=0):\n", df.count(axis=0), "\n")
print("count(axis=1):\n", df.count(axis=1), "\n")

#  quantile
print("quantile:\n", df.quantile([0.25, 0.5, 0.75], numeric_only=True))
```

Vishal Acharya

```
DataFrame:
   Name  Marks  Age
0    A     85   20
1    B     90   21
2    C     90   21

sum(axis=0):
 Marks    265
Age       62
dtype: int64

mean(axis=0):
 Marks    88.333333
Age      20.666667
dtype: float64

max(axis=0):
 Marks    90
Age      21
dtype: int64

min(axis=0):
 Marks    85
Age      20
dtype: int64

median(axis=0):
 Marks    90.0
Age      21.0
dtype: float64

mode(axis=0):
    Marks  Age
0     90   21

sum(axis=1):
 0    105
1    111
2    111
dtype: int64

mean(axis=1):
 0    52.5
1    55.5
2    55.5
dtype: float64

max(axis=1):
 0    85
1    90
2    90
dtype: int64

min(axis=1):
 0    20
1    21
2    21
dtype: int64

median(axis=1):
```

Vishal Acharya

```
 0    52.5
1    55.5
2    55.5
dtype: float64

mode(axis=1):
     0   1
0  20  85
1  21  90
2  21  90

count(axis=0):
 Name     3
Marks    3
Age      3
dtype: int64

count(axis=1):
 0    3
1    3
2    3
dtype: int64

quantile:
       Marks   Age
0.25   87.5  20.5
0.50   90.0  21.0
0.75   90.0  21.0
```

df.corr(numeric_only=True) → Returns the correlation matrix of numeric columns (shows relationship between variables from -1 to +1)

In [4]:
```python
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'Marks': [85, 90, 95, 100],
    'Age': [20, 21, 22, 23],
    'Height': [150, 155, 160, 165],
    'Name': ['A', 'B', 'C', 'D']
})

# Correlation
print(df.corr(numeric_only=True))
```

```
        Marks  Age  Height
Marks     1.0  1.0     1.0
Age       1.0  1.0     1.0
Height    1.0  1.0     1.0
```

d.plotting.scatter_matrix(frame, alpha=0.5, figsize=None) → Creates a matrix of scatter plots to visualize relationships between all numeric columns

frame → DataFrame containing data

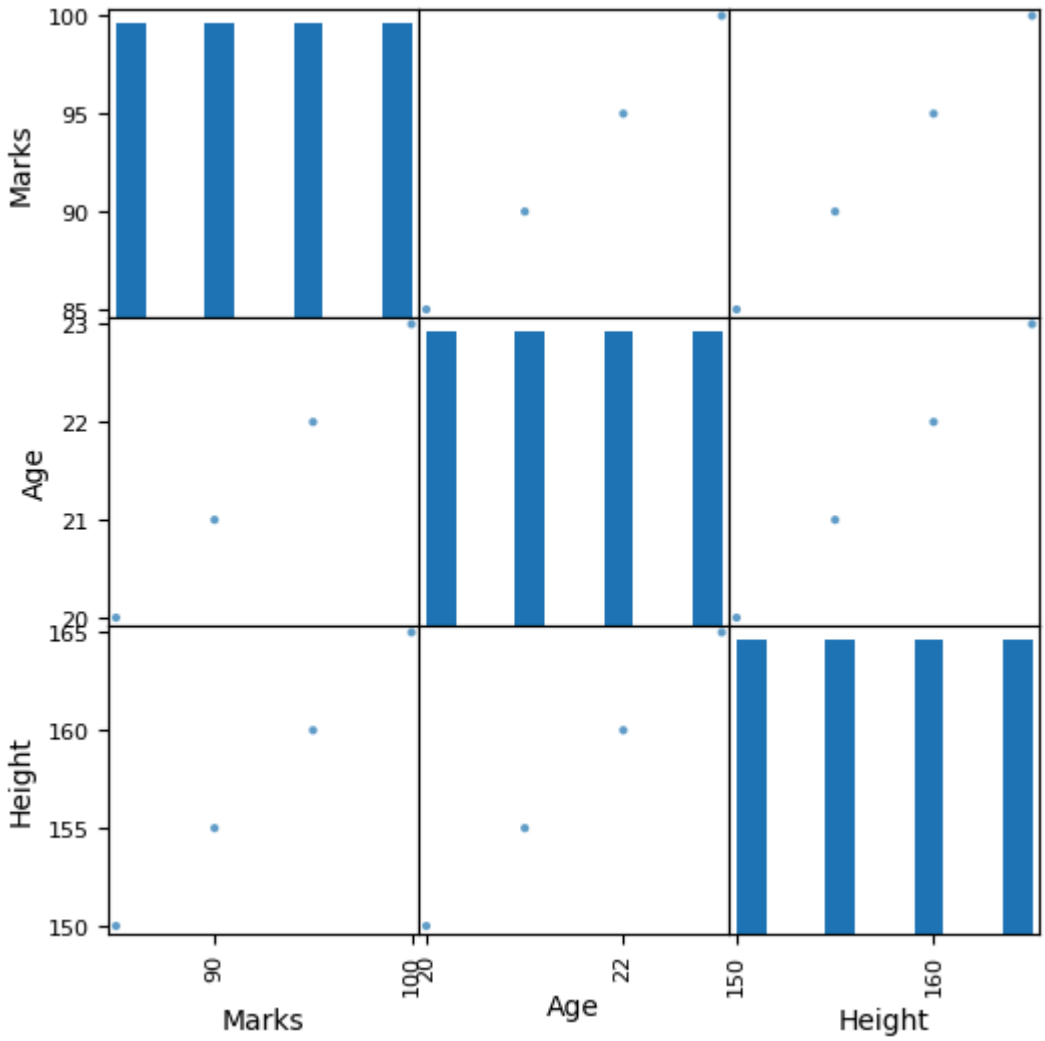
alpha → Transparency of points (0 to 1)

Vishal Acharya

figsize → Size of the entire plot

Helps identify correlation, trends, and patterns between multiple variables

In [5]:
```
import pandas as pd
import matplotlib.pyplot as plt
# Create DataFrame
df = pd.DataFrame({
    'Marks': [85, 90, 95, 100],
    'Age': [20, 21, 22, 23],
    'Height': [150, 155, 160, 165]
})
# Scatter matrix
pd.plotting.scatter_matrix(df, alpha=0.7, figsize=(6, 6))
plt.show()
```



pd.plotting.parallel_coordinates(frame, class_column, cols=None, color=None) → Plots multi-dimensional data as parallel lines grouped by a class/category

frame → DataFrame containing data

class_column → Column used to group/classify lines

cols → Specific columns to plot (optional)

Vishal Acharya

color → Colors for different classes

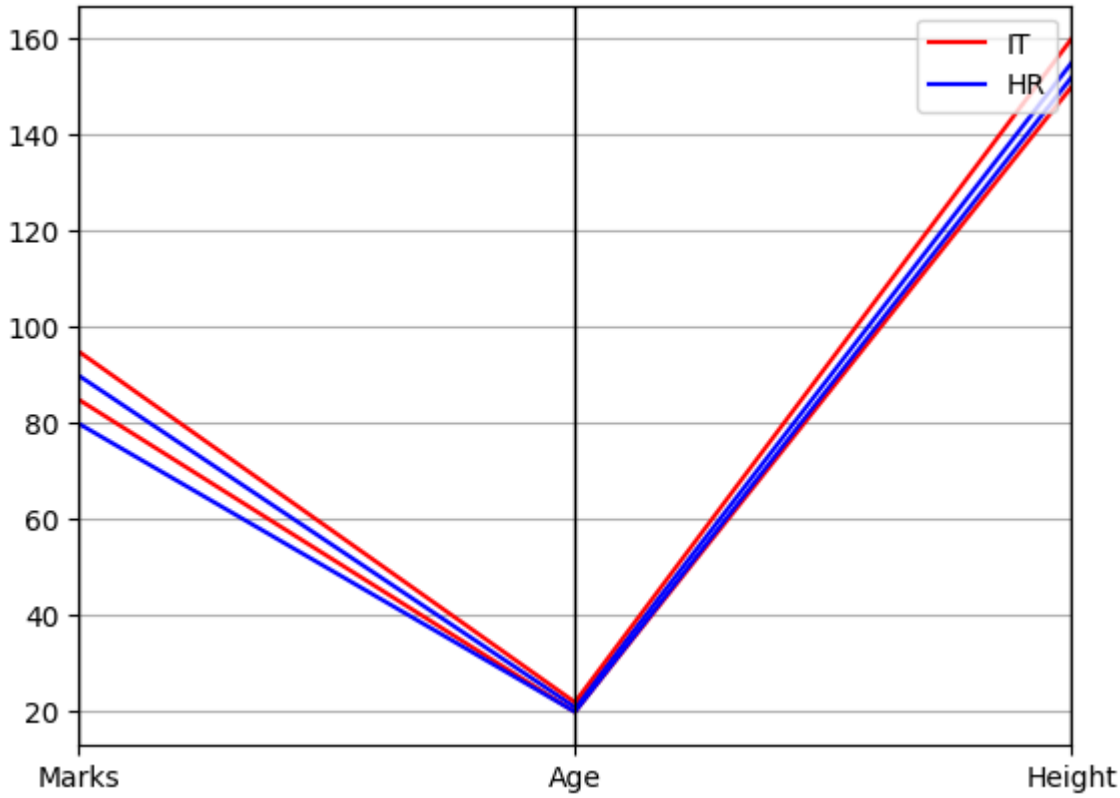
Compare multiple features across categories

Useful for pattern recognition & classification Each row → one line

X-axis → features (Marks, Age, Height)

Colors → different classes (Dept)

In [7]:
```
import pandas as pd
import matplotlib.pyplot as plt
from pandas.plotting import parallel_coordinates
# Create DataFrame
df = pd.DataFrame({
    'Marks': [85, 90, 95, 80],
    'Age': [20, 21, 22, 20],
    'Height': [150, 155, 160, 152],
    'Dept': ['IT', 'HR', 'IT', 'HR']
})
# Parallel Coordinates Plot
parallel_coordinates(df, class_column='Dept', color=['red', 'blue'])
plt.show()
```



pd.crosstab(index, columns, margins=False) → Creates a frequency table showing count of combinations between two variables

index → Values for rows (first variable)

columns → Values for columns (second variable)

Vishal Acharya

margins → If True, adds totals (row & column sums)

Analyze relationship between two categorical variables

In [8]:
```
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'Dept': ['IT', 'HR', 'IT', 'Finance', 'HR', 'IT'],
    'Gender': ['M', 'F', 'F', 'M', 'F', 'M']
})

# Crosstab without totals
print("Crosstab:\n", pd.crosstab(df['Dept'], df['Gender']), "\n")

# Crosstab with totals
print("Crosstab with margins:\n",
      pd.crosstab(df['Dept'], df['Gender'], margins=True))
```

```
Crosstab:
 Gender   F  M
Dept
Finance  0  1
HR       2  0
IT       1  2

Crosstab with margins:
 Gender   F  M  All
Dept
Finance  0  1    1
HR       2  0    2
IT       1  2    3
All      3  3    6
```

df.apply(func, axis=0, raw=False, result_type=None, args=(), **kwargs) → Applies a function along rows or columns of a DataFrame

func → Function to apply on data

axis → 0 = apply on columns, 1 = apply on rows

raw → If True, passes NumPy array instead of Series

result_type → Controls output shape ('expand', 'reduce', 'broadcast')

args → Extra positional arguments for function

**kwargs → Extra keyword arguments

apply() = flexible function application

Works on rows or columns

Can use built-in or custom functions

Vishal Acharya

In [9]:
```
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'A': [10, 20, 30],
    'B': [1, 2, 3]
})

print("DataFrame:\n", df, "\n")

#  🔹 axis=0 → apply on columns
print("Column sum using apply:\n",
      df.apply(sum, axis=0), "\n")

#  🔹 axis=1 → apply on rows
print("Row sum using apply:\n",
      df.apply(sum, axis=1), "\n")

#  🔹 using custom function
def square(x):
    return x * x

print("Apply custom function:\n",
      df.apply(square), "\n")

#  🔹 using args
def add_value(x, val):
    return x + val

print("Apply with args:\n",
      df.apply(add_value, args=(5,)))
```

Vishal Acharya

```
DataFrame:
     A  B
0  10  1
1  20  2
2  30  3

Column sum using apply:
 A    60
B     6
dtype: int64

Row sum using apply:
 0    11
1    22
2    33
dtype: int64

Apply custom function:
      A  B
0  100  1
1  400  4
2  900  9

Apply with args:
     A  B
0  15  6
1  25  7
2  35  8
```

In [10]:

```python
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'A': [10, 20, 30],
    'B': [1, 2, 3]
})
print("DataFrame:\n", df, "\n")
# Lambda function → multiply each value by 2
print("Using lambda function:\n",
      df.apply(lambda x: x * 2), "\n")

#  kwargs example
def add_values(x, add1=0, add2=0):
    return x + add1 + add2

print("Using kwargs (add1=5, add2=10):\n",
      df.apply(add_values, add1=5, add2=10))
```

Vishal Acharya

```
DataFrame:
     A  B
0  10  1
1  20  2
2  30  3

Using lambda function:
     A  B
0  20  2
1  40  4
2  60  6

Using kwargs (add1=5, add2=10):
     A   B
0  25  16
1  35  17
2  45  18
```

df.astype(dtype) → Converts data type of entire DataFrame or specific columns to the given type

dtype → Target data type (e.g., int, float, str, or dictionary for specific columns)

Convert all columns → df.astype(float)

Convert specific column → df.astype({'A': 'int'})

In [11]:
```
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4.5, 5.5, 6.5]
})

print("Original dtypes:\n", df.dtypes, "\n")

#  🔹 Convert entire DataFrame to float
df1 = df.astype(float)
print("After converting all to float:\n", df1.dtypes, "\n")

#  🔹 Convert specific column
df2 = df.astype({'A': 'float'})
print("After converting column A:\n", df2.dtypes)
```

Vishal Acharya

```
Original dtypes:
 A      int64
B    float64
dtype: object

After converting all to float:
 A    float64
B    float64
dtype: object

After converting column A:
 A    float64
B    float64
dtype: object
```

df.replace(to_replace, value, inplace=False) → Replaces specified values in DataFrame with new values

to_replace=old valuee

value= new value

Dictionary → {old_value: new_value}

In [13]:
```
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'Name': ['A', 'B', 'C'],
    'Marks': [85, 90, 95]
})

print("Original DataFrame:\n", df, "\n")
#simple replace
df_new=df.replace(95,105)
print("After replace:\n", df_new)
# Replace using dictionary
df_new = df.replace({'A': 'X', 90: 100})

print("After replace:\n", df_new)
#replace on column
print("replace on column")
print(df["Name"].replace("C","M"))
```

Vishal Acharya

```
Original DataFrame:
   Name  Marks
0    A     85
1    B     90
2    C     95

After replace:
   Name  Marks
0    A     85
1    B     90
2    C    105
After replace:
   Name  Marks
0    X     85
1    B    100
2    C     95
replace on column
0    A
1    B
2    M
Name: Name, dtype: object
```

pd.concat(objs, axis=0, join='outer', ignore_index=False, sort=False)

objs → List of DataFrames/Series to concatenate

axis → 0 = stack rows, 1 = join columns.0 → vertical join,xis=1 → horizontal join

join='outer' → Keeps all columns (union), fills missing with NaN → all columns (NaN allowed)

join='inner' → Keeps only common columns (intersection)→ only common columns

ignore_index → If True, resets index (0,1,2,...)

sort → Sorts column names if needed

In [1]:
```python
import pandas as pd

# Create DataFrames
df1 = pd.DataFrame({
    'A': [1, 2],
    'B': [3, 4]
})

df2 = pd.DataFrame({
    'B': [5, 6],
    'C': [7, 8]
})

print("DF1:\n", df1, "\n")
print("DF2:\n", df2, "\n")

#  ◆  axis=0 with outer (default)
print("Concat axis=0 (outer):\n",
      pd.concat([df1, df2], axis=0), "\n")
```

Vishal Acharya

```python
# 🔹 axis=0 with inner
print("Concat axis=0 (inner):\n",
      pd.concat([df1, df2], axis=0, join='inner'), "\n")

# 🔹 axis=1 (column-wise outer)
print("Concat axis=1 (outer):\n",
      pd.concat([df1, df2], axis=1), "\n")

# 🔹 axis=1 with inner
print("Concat axis=1 (inner):\n",
      pd.concat([df1, df2], axis=1, join='inner'), "\n")

# 🔹 ignore_index=True
print("Concat with ignore_index:\n",
      pd.concat([df1, df2], ignore_index=True), "\n")

# 🔹 keys (add hierarchical index)
print("Concat with keys:\n",
      pd.concat([df1, df2], keys=['First', 'Second']), "\n")

# 🔹 sort=True (sort column names)
print("Concat with sort=True:\n",
      pd.concat([df1, df2], sort=True), "\n")

# 🔹 different index example
df3 = pd.DataFrame({
    'A': [9, 10]
}, index=[5, 6])

print("DF3:\n", df3, "\n")

print("Concat df1 & df3 (axis=1):\n",
      pd.concat([df1, df3], axis=1), "\n")
```

Vishal Acharya

```
DF1:
    A  B
0  1  3
1  2  4

DF2:
    B  C
0  5  7
1  6  8

Concat axis=0 (outer):
      A  B    C
0  1.0  3  NaN
1  2.0  4  NaN
0  NaN  5  7.0
1  NaN  6  8.0

Concat axis=0 (inner):
    B
0  3
1  4
0  5
1  6

Concat axis=1 (outer):
    A  B  B  C
0  1  3  5  7
1  2  4  6  8

Concat axis=1 (inner):
    A  B  B  C
0  1  3  5  7
1  2  4  6  8

Concat with ignore_index:
      A  B    C
0  1.0  3  NaN
1  2.0  4  NaN
2  NaN  5  7.0
3  NaN  6  8.0

Concat with keys:
             A  B    C
First  0  1.0  3  NaN
       1  2.0  4  NaN
Second 0  NaN  5  7.0
       1  NaN  6  8.0

Concat with sort=True:
      A  B    C
0  1.0  3  NaN
1  2.0  4  NaN
0  NaN  5  7.0
1  NaN  6  8.0

DF3:
     A
5   9
6  10
```

Vishal Acharya

```
Concat df1 & df3 (axis=1):
      A    B     A
0  1.0  3.0   NaN
1  2.0  4.0   NaN
5  NaN  NaN   9.0
6  NaN  NaN  10.0
```

pd.merge(left, right, how='inner', on=None, left_on=None, right_on=None, left_index=False, right_index=False, sort=False, suffixes=('_x', '_y'))

left, right → DataFrames to merge

how → Type of join: 'inner', 'left', 'right', 'outer'

on → Common column name(s) in both DataFrames

left_on / right_on → Column names if different in both DataFrames

left_index / right_index → Use index instead of columns for joining

sort → Sort result by join keys

inner → common values only

left → all from left + matching right

right → all from right + matching left

outer → all values (union)

on vs left_on/right_on → same vs different column names

index join → use index instead of column suffixes → Rename overlapping columns (_x, _y)

In [15]:

```python
import pandas as pd

# Create DataFrames
df1 = pd.DataFrame({
    'ID': [1, 2, 3],
    'Name': ['A', 'B', 'C']
})

df2 = pd.DataFrame({
    'ID': [2, 3, 4],
    'Marks': [90, 95, 85]
})

print("DF1:\n", df1, "\n")
print("DF2:\n", df2, "\n")

#  🔹 INNER JOIN (common values)
print("Inner Join:\n",
      pd.merge(df1, df2, on='ID', how='inner'), "\n")

#  🔹 LEFT JOIN
print("Left Join:\n",
```

Vishal Acharya

```
      pd.merge(df1, df2, on='ID', how='left'), "\n")

#  ◆  RIGHT JOIN
print("Right Join:\n",
      pd.merge(df1, df2, on='ID', how='right'), "\n")

#  ◆  OUTER JOIN
print("Outer Join:\n",
      pd.merge(df1, df2, on='ID', how='outer'), "\n")

#  ◆  Different column names (left_on, right_on)
df3 = pd.DataFrame({
    'Key': [1, 2, 3],
    'City': ['X', 'Y', 'Z']
})

print("Merge with different column names:\n",
      pd.merge(df1, df3, left_on='ID', right_on='Key'), "\n")

#  ◆  Using index
df1_index = df1.set_index('ID')
df2_index = df2.set_index('ID')

print("Merge using index:\n",
      pd.merge(df1_index, df2_index,
               left_index=True, right_index=True), "\n")

#  ◆  Overlapping column names (suffixes)
df4 = pd.DataFrame({
    'ID': [1, 2, 3],
    'Marks': [50, 60, 70]
})

print("Merge with suffixes:\n",
      pd.merge(df2, df4, on='ID', suffixes=('_df2', '_df4')))
```

Vishal Acharya

```
DF1:
    ID Name
0   1    A
1   2    B
2   3    C

DF2:
    ID  Marks
0   2     90
1   3     95
2   4     85

Inner Join:
    ID Name  Marks
0   2    B     90
1   3    C     95

Left Join:
    ID Name  Marks
0   1    A    NaN
1   2    B   90.0
2   3    C   95.0

Right Join:
    ID Name  Marks
0   2    B     90
1   3    C     95
2   4  NaN     85

Outer Join:
    ID Name  Marks
0   1    A    NaN
1   2    B   90.0
2   3    C   95.0
3   4  NaN   85.0

Merge with different column names:
    ID Name  Key City
0   1    A    1    X
1   2    B    2    Y
2   3    C    3    Z

Merge using index:
    Name  Marks
ID
2     B     90
3     C     95

Merge with suffixes:
    ID  Marks_df2  Marks_df4
0   2         90         60
1   3         95         70
```

df.groupby() → Splits data into groups and applies operations on each group

1. Group → split data

Vishal Acharya

2. Apply → perform operation
3. Combine → get result

- by → Column(s) used for grouping
- axis=0 → Group row-wise
- as_index=True → Group labels become index
- sort=True → Sort group keys

loop → iterate groups

sample() → random rows per group

size() → number of rows per group

nth(n) → nth row from each group

groups → dictionary of group indices

nunique() → unique values per group

count() → non-null count

sum(), mean(), median() → statistics

agg() → multiple aggregations

apply() → custom function

describe() → summary stats

value_counts() → frequency per group

sort_values() → sorting

In [2]:
```
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'Dept': ['IT','IT','HR','HR','Finance','Finance'],
    'Employee': ['A','B','C','D','E','F'],
    'Salary': [50000,60000,45000,47000,70000,72000],
    'Experience': [2,5,3,7,4,6]
})

print("DataFrame:\n", df, "\n")

# Groupby object
g = df.groupby('Dept')

#  🔹 loop through groups
for name, group in g:
    print("Group:", name)
    print(group, "\n")

#  🔹 sample
print("Sample:\n", g.sample(1), "\n")
```

Vishal Acharya

```python
# 🔹 size
print("Size:\n", g.size(), "\n")

# 🔹 nth
print("Nth (first row of each group):\n", g.nth(0), "\n")

# 🔹 groups
print("Groups:\n", g.groups, "\n")

# 🔹 nunique
print("Unique count:\n", g.nunique(), "\n")

# 🔹 count
print("Count:\n", g.count(), "\n")

# 🔹 sum
print("Sum:\n", g.sum(numeric_only=True), "\n")

# 🔹 mean
print("Mean:\n", g.mean(numeric_only=True), "\n")

# 🔹 median
print("Median:\n", g.median(numeric_only=True), "\n")

# 🔹 agg
print("Agg:\n", g.agg({'Salary':['min','max'], 'Experience':'mean'}), "\n")

# 🔹 apply
print("Apply:\n", g.apply(lambda x: x['Salary'].max()), "\n")

# 🔹 describe
print("Describe:\n", g.describe(), "\n")

# 🔹 value_counts
print("Value Counts:\n", df.groupby('Dept')['Employee'].value_counts(), "\n")

# 🔹 sort_values
print("Sort within group:\n", df.sort_values(['Dept','Salary']))
```

Vishal Acharya

```
DataFrame:
       Dept Employee  Salary  Experience
0        IT        A   50000           2
1        IT        B   60000           5
2        HR        C   45000           3
3        HR        D   47000           7
4   Finance        E   70000           4
5   Finance        F   72000           6

Group: Finance
      Dept Employee  Salary  Experience
4   Finance        E   70000           4
5   Finance        F   72000           6

Group: HR
  Dept Employee  Salary  Experience
2   HR        C   45000           3
3   HR        D   47000           7

Group: IT
  Dept Employee  Salary  Experience
0   IT        A   50000           2
1   IT        B   60000           5

Sample:
       Dept Employee  Salary  Experience
4   Finance        E   70000           4
3        HR        D   47000           7
0        IT        A   50000           2

Size:
 Dept
Finance    2
HR         2
IT         2
dtype: int64

Nth (first row of each group):
       Dept Employee  Salary  Experience
0        IT        A   50000           2
2        HR        C   45000           3
4   Finance        E   70000           4

Groups:
 {'Finance': [4, 5], 'HR': [2, 3], 'IT': [0, 1]}

Unique count:
          Employee  Salary  Experience
Dept
Finance          2       2           2
HR               2       2           2
IT               2       2           2

Count:
          Employee  Salary  Experience
Dept
Finance          2       2           2
HR               2       2           2
IT               2       2           2
```

Vishal Acharya

```
Sum:
          Salary  Experience
Dept
Finance  142000          10
HR        92000          10
IT       110000           7

Mean:
           Salary  Experience
Dept
Finance  71000.0         5.0
HR       46000.0         5.0
IT       55000.0         3.5

Median:
           Salary  Experience
Dept
Finance  71000.0         5.0
HR       46000.0         5.0
IT       55000.0         3.5

Agg:
         Salary        Experience
           min    max       mean
Dept
Finance  70000  72000        5.0
HR       45000  47000        5.0
IT       50000  60000        3.5

Apply:
 Dept
Finance    72000
HR         47000
IT         60000
dtype: int64

Describe:
         Salary                                                                \
         count     mean          std      min      25%      50%      75%
Dept
Finance    2.0  71000.0  1414.213562  70000.0  70500.0  71000.0  71500.0
HR         2.0  46000.0  1414.213562  45000.0  45500.0  46000.0  46500.0
IT         2.0  55000.0  7071.067812  50000.0  52500.0  55000.0  57500.0

                Experience
             max      count mean       std  min   25%  50%   75%  max
Dept
Finance  72000.0        2.0  5.0  1.414214  4.0  4.50  5.0  5.50  6.0
HR       47000.0        2.0  5.0  2.828427  3.0  4.00  5.0  6.00  7.0
IT       60000.0        2.0  3.5  2.121320  2.0  2.75  3.5  4.25  5.0

Value Counts:
 Dept     Employee
Finance  E           1
         F           1
HR       C           1
         D           1
IT       A           1
         B           1
Name: count, dtype: int64
```

Vishal Acharya

```
Sort within group:
       Dept Employee  Salary  Experience
4  Finance        E   70000           4
5  Finance        F   72000           6
2       HR        C   45000           3
3       HR        D   47000           7
0       IT        A   50000           2
1       IT        B   60000           5
```

```
C:\Users\VISHAL\AppData\Local\Temp\ipykernel_1072\2251304101.py:52: FutureWarnin
g: DataFrameGroupBy.apply operated on the grouping columns. This behavior is depr
ecated, and in a future version of pandas the grouping columns will be excluded f
rom the operation. Either pass `include_groups=False` to exclude the groupings or
explicitly select the grouping columns after groupby to silence this warning.
  print("Apply:\n", g.apply(lambda x: x['Salary'].max()), "\n")
```

In [3]:

```python
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'Dept': ['IT','IT','HR','HR','Finance','Finance'],
    'Gender': ['M','F','M','F','M','F'],
    'Salary': [50000,60000,45000,47000,70000,72000]
})

print("DataFrame:\n", df, "\n")

# Group by two columns
g = df.groupby(['Dept', 'Gender'])

# 🔹 Sum
print("Sum:\n", g.sum(numeric_only=True), "\n")

# 🔹 Mean
print("Mean:\n", g.mean(numeric_only=True), "\n")

# 🔹 Count
print("Count:\n", g.count(), "\n")

# 🔹 Size
print("Size:\n", g.size(), "\n")

# 🔹 Aggregation
print("Agg:\n", g.agg({'Salary':['min','max','mean']}))
```

Vishal Acharya

```
DataFrame:
       Dept Gender  Salary
0       IT      M   50000
1       IT      F   60000
2       HR      M   45000
3       HR      F   47000
4  Finance      M   70000
5  Finance      F   72000

Sum:
                 Salary
Dept    Gender
Finance F        72000
        M        70000
HR      F        47000
        M        45000
IT      F        60000
        M        50000

Mean:
                  Salary
Dept    Gender
Finance F       72000.0
        M       70000.0
HR      F       47000.0
        M       45000.0
IT      F       60000.0
        M       50000.0

Count:
                 Salary
Dept    Gender
Finance F            1
        M            1
HR      F            1
        M            1
IT      F            1
        M            1

Size:
 Dept     Gender
Finance  F         1
         M         1
HR       F         1
         M         1
IT       F         1
         M         1
dtype: int64

Agg:
                Salary
                  min    max     mean
Dept    Gender
Finance F       72000  72000  72000.0
        M       70000  70000  70000.0
HR      F       47000  47000  47000.0
        M       45000  45000  45000.0
IT      F       60000  60000  60000.0
        M       50000  50000  50000.0
```

Vishal Acharya

seaborn.heatmap() → Visualizes data as a color-coded matrix to show patterns, correlations, or intensity

data → Input 2D data (DataFrame or matrix)

vmin / vmax → Set minimum and maximum value range for color scaling

cmap → Color map (e.g., 'coolwarm', 'viridis')

linewidths → Width of lines between cells

linecolor → Color of cell borders

annot → Shows values inside cells if True

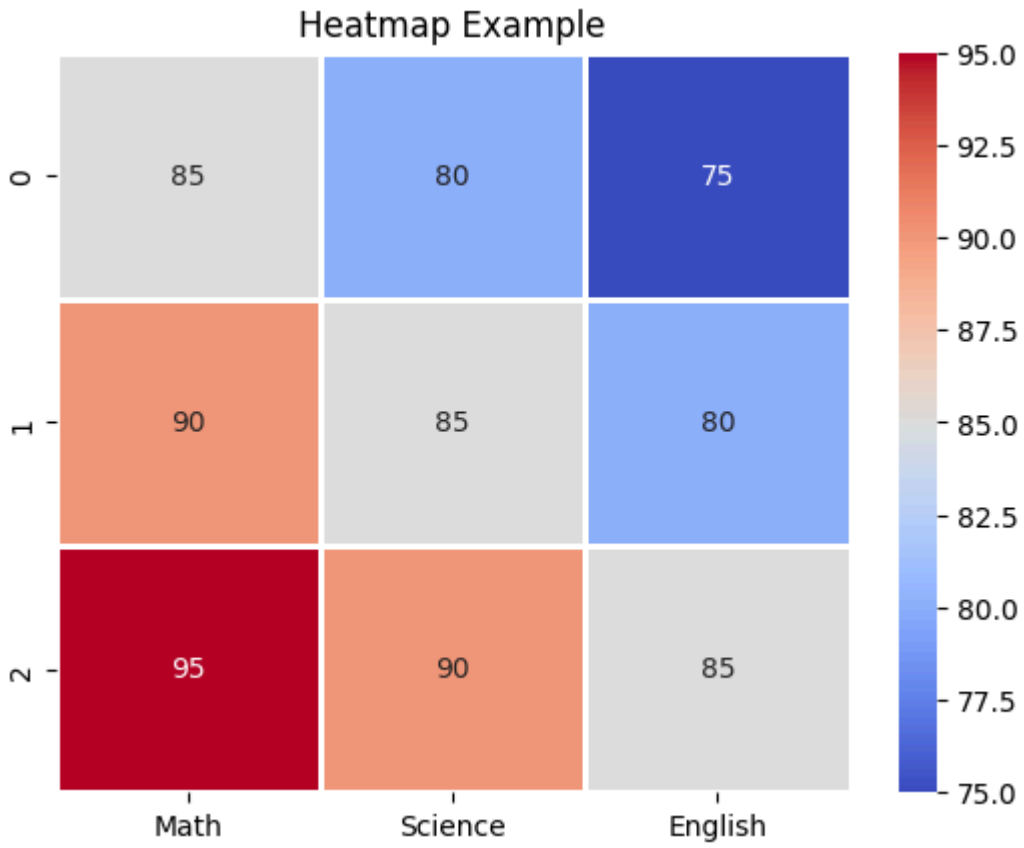
cbar → Displays color bar (legend)

Shows correlation matrix, patterns, intensity differences

Dark/light colors → high/low values

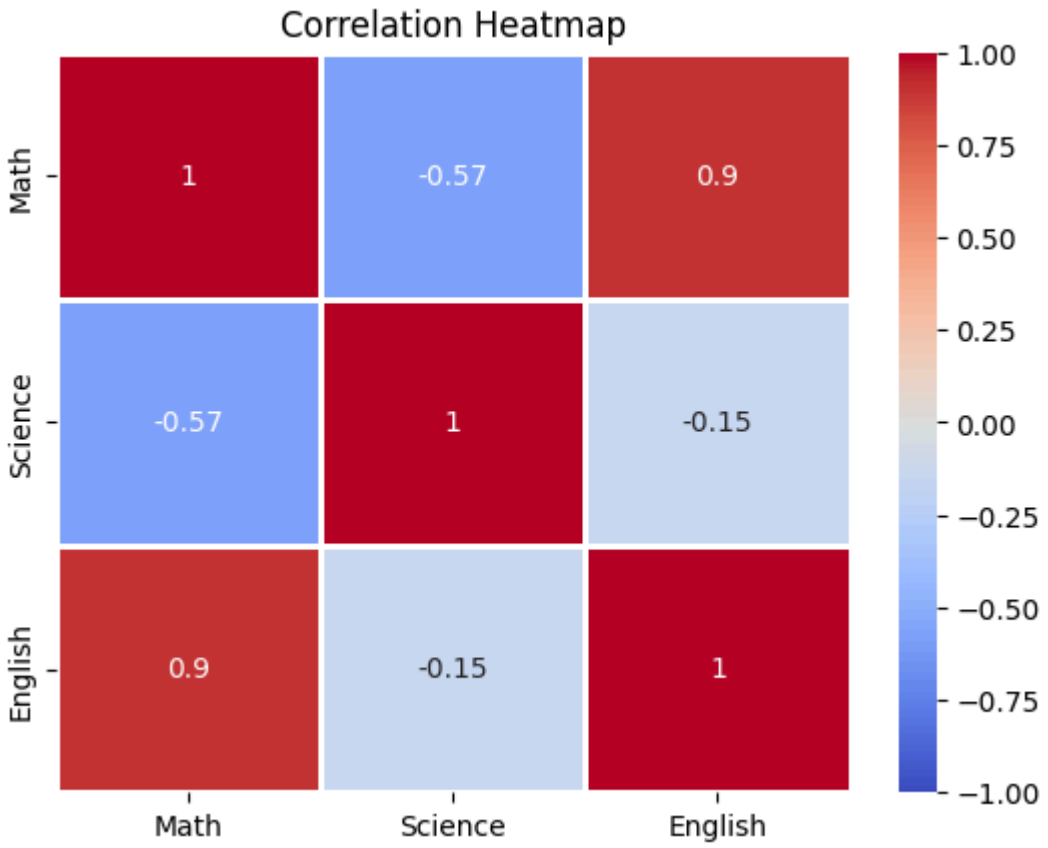
In [19]:
```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

# Create DataFrame
df = pd.DataFrame({
    'Math': [85, 90, 95],
    'Science': [80, 85, 90],
    'English': [75, 80, 85]
})

# Heatmap
sns.heatmap(df,
            cmap='coolwarm',
            annot=True,
            linewidths=1,
            linecolor='white')

plt.title("Heatmap Example")
plt.show()
```

Vishal Acharya



In [22]:
```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

# Create DataFrame
df = pd.DataFrame({
    'Math': [85, 90, 95],
    'Science': [60, 25, 40],
    'English': [15, 20, 85]
})

# Correlation Heatmap with all parameters
sns.heatmap(
    df.corr(),          # data
    vmin=-1,            # minimum value
    vmax=1,             # maximum value
    cmap='coolwarm',    # color map
    linewidths=1,       # line width between cells
    linecolor='white',  # line color
    annot=True,         # show values
    cbar=True           # show color bar
)

plt.title("Correlation Heatmap")
plt.show()
```

Vishal Acharya



Vishal Acharya

sns.boxplot(x, y, hue, data, color) → Creates a box plot to show distribution, median, and outliers of data

Shows median, quartiles, and outliers

Useful for IQR and outlier detection

x → Categorical variable (groups on x-axis)

y → Numeric variable (values to analyze)

hue → Sub-grouping within categories (adds multiple boxes)

data → DataFrame containing data

color → Color of the box plot

In [25]:
```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

# Create larger DataFrame
df = pd.DataFrame({
    'Dept': ['IT']*10 + ['HR']*10 + ['Finance']*10,
    'Gender': ['M','F']*15,
    'Salary': [
        # IT salaries
```

```
        50000, 55000, 52000, 58000, 60000, 61000, 53000, 57000, 59000, 62000,
        # HR salaries
        45000, 47000, 46000, 48000, 49000, 50000, 45500, 46500, 47500, 48500,
        # Finance salaries
        60000, 62000, 61000, 63000, 65000, 64000, 60500, 61500, 62500, 66000
    ]
})

# Boxplot
sns.boxplot(
    x='Dept',
    y='Salary',
    hue='Gender',
    data=df,
    palette='Set2'   # better colors for groups
)

plt.title("Salary Distribution by Department")
plt.show()
```

Vishal Acharya

sns.scatterplot(x, y, hue, style, s, data) → Creates a scatter plot to show relationship between two variables with grouping by color and style hows relationship (correlation) between variables

x → Values for x-axis

y → Values for y-axis

hue → Color grouping (different categories)

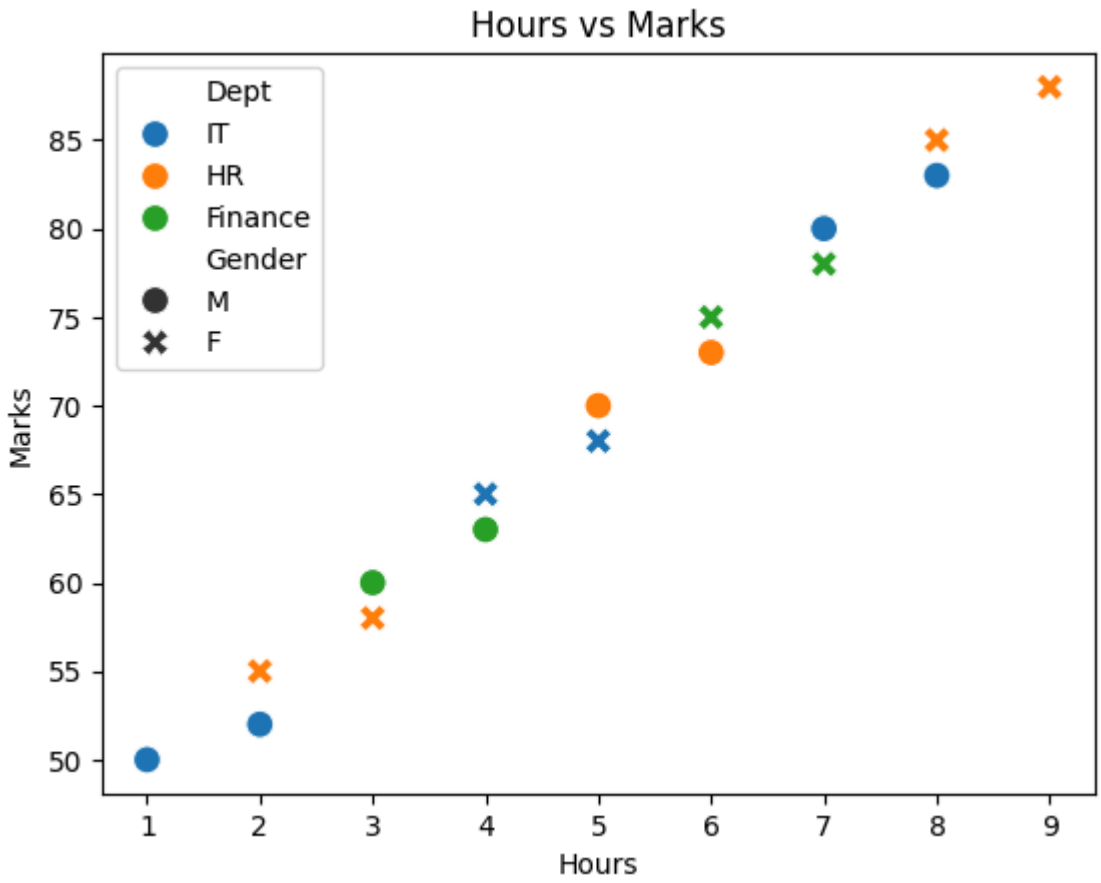
style → Marker style based on category(shape grouping)

s → Size of points

data → DataFrame

In [26]:
```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

# Create larger dataset
df = pd.DataFrame({
    'Hours': [1,2,3,4,5,6,7,8,2,3,4,5,6,7,8,9],
    'Marks': [50,55,60,65,70,75,80,85,52,58,63,68,73,78,83,88],
    'Gender': ['M','F']*8,
    'Dept': ['IT','HR','Finance','IT','HR','Finance','IT','HR']*2
})

# Scatter plot
sns.scatterplot(
    x='Hours',
    y='Marks',
    hue='Dept',       # color grouping
    style='Gender',   # marker style
    s=100,            # size of points
    data=df
)

plt.title("Hours vs Marks")
plt.show()
```

Vishal Acharya



Vishal Acharya

Networkx

nx.Graph() → Creates undirected graph

nx.DiGraph() → Creates directed graph

add_node() → Adds single node

add_nodes_from() → Adds multiple nodes

add_edge(u,v) → Adds edge between nodes

add_edges_from() → Adds multiple edges

self-loop → Edge from node to itself (('A','A'))

G → Graph object to draw

with_labels → Show node labels if True

node_size → Size of nodes

node_color → Color of nodes

width → Thickness of edges

edge_color → Color of edges

arrows → Show direction arrows (for directed graph)

nx.draw() → Visualizes graph

Undirected → no direction

Directed → arrows show direction

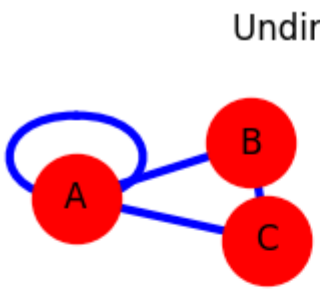
Loop → node connected to itself

In [27]:
```
import networkx as nx
import matplotlib.pyplot as plt

#  ◆  1. Undirected Graph
G = nx.Graph()

# Add nodes
G.add_node(1)
G.add_nodes_from(['A', 'B', 'C'])

# Add edges
G.add_edge(3, 1)
G.add_edges_from([('A', 'B'), ('A', 'C'), ('B', 'C')])

# Add self-loop
G.add_edge('A', 'A')

print("Undirected Graph Nodes:", G.nodes())
print("Undirected Graph Edges:", G.edges())

# Draw Undirected Graph
plt.figure(figsize=(5,5))
nx.draw(G, with_labels=True, node_size=1000, node_color='red',
        width=3, edge_color='blue')
plt.title("Undirected Graph with Loop")
plt.show()


#  ◆  2. Directed Graph
DG = nx.DiGraph()

# Add nodes
DG.add_nodes_from(['X', 'Y', 'Z'])

# Add directed edges
DG.add_edge('X', 'Y')
DG.add_edge('Y', 'Z')
DG.add_edge('Z', 'X')

# Add self-loop
DG.add_edge('X', 'X')

print("Directed Graph Nodes:", DG.nodes())
print("Directed Graph Edges:", DG.edges())

# Draw Directed Graph
plt.figure(figsize=(5,5))
nx.draw(DG, with_labels=True, node_size=1000, node_color='green',
        width=3, edge_color='black', arrows=True)
```

Vishal Acharya

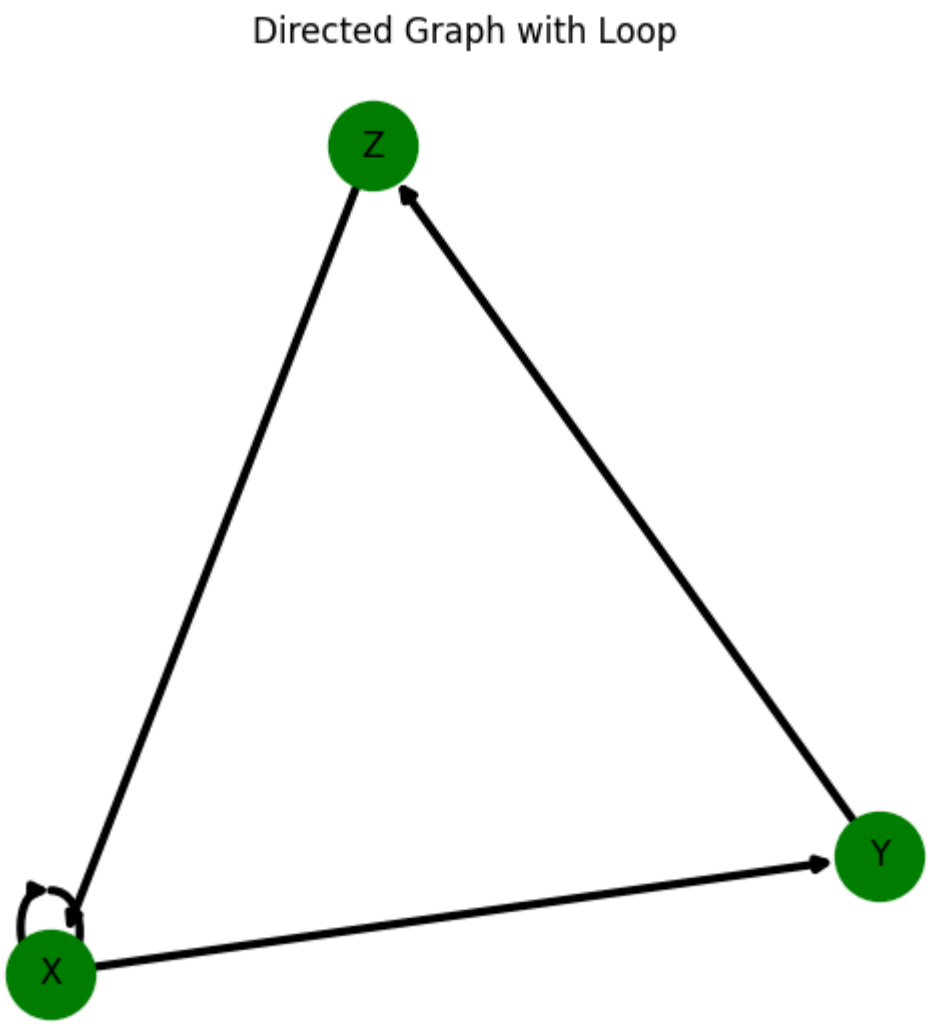
```
plt.title("Directed Graph with Loop")
plt.show()
```

```
Undirected Graph Nodes: [1, 'A', 'B', 'C', 3]
Undirected Graph Edges: [(1, 3), ('A', 'B'), ('A', 'C'), ('A', 'A'), ('B', 'C')]
```



```
Directed Graph Nodes: ['X', 'Y', 'Z']
Directed Graph Edges: [('X', 'Y'), ('X', 'X'), ('Y', 'Z'), ('Z', 'X')]
```

Vishal Acharya

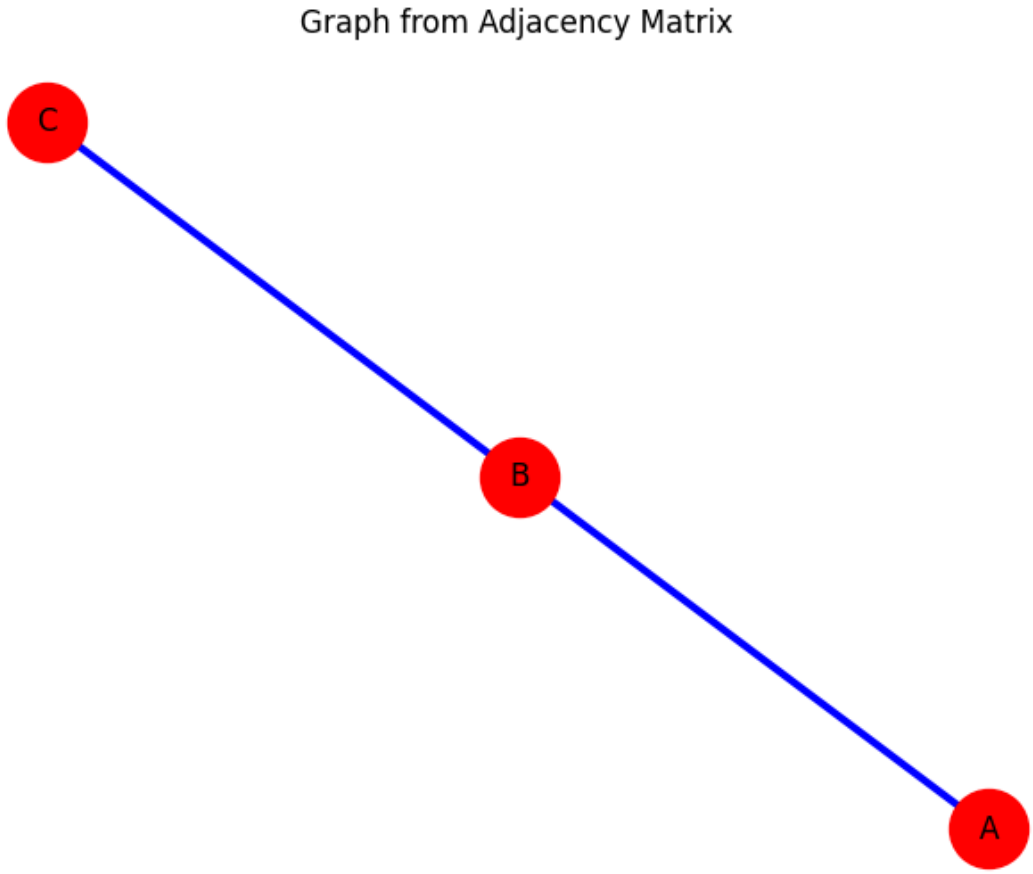


Vishal Acharya

In [28]:
```
import networkx as nx
import matplotlib.pyplot as plt

# Given matrix
matrix = [
    [0, 1, 0],
    [1, 0, 1],
    [0, 1, 0]
]

nodes = ['A', 'B', 'C']

# Create graph
G = nx.Graph()

# Add nodes
G.add_nodes_from(nodes)

# Add edges using loop
for i in range(len(matrix)):
    for j in range(i+1, len(matrix)):   # avoid duplicate edges
        if matrix[i][j] == 1:
            G.add_edge(nodes[i], nodes[j])

# Print edges
print("Edges:", G.edges())
```

```
# Draw graph
nx.draw(G, with_labels=True, node_size=1000,
        node_color='red', edge_color='blue', width=3)

plt.title("Graph from Adjacency Matrix")
plt.show()
```

```
Edges: [('A', 'B'), ('B', 'C')]
```



Vishal Acharya

px.bar() → Creates interactive bar charts to compare values across categories

x / y → Axes values

width / height → Size of chart

title → Chart title

color → Grouping by category (different colors)

hover_data → Extra info on hover

pattern_shape → Pattern style (/, x, .)

barmode → 'group', 'stack', 'relative'

text_auto / text → Show values on bars

orientation → 'v' (vertical), 'h' (horizontal)

color_discrete_map → Custom color mapping

color_discrete_sequence → List of colors

Simple → single comparison

Group → compare categories across groups

Stack → total contribution

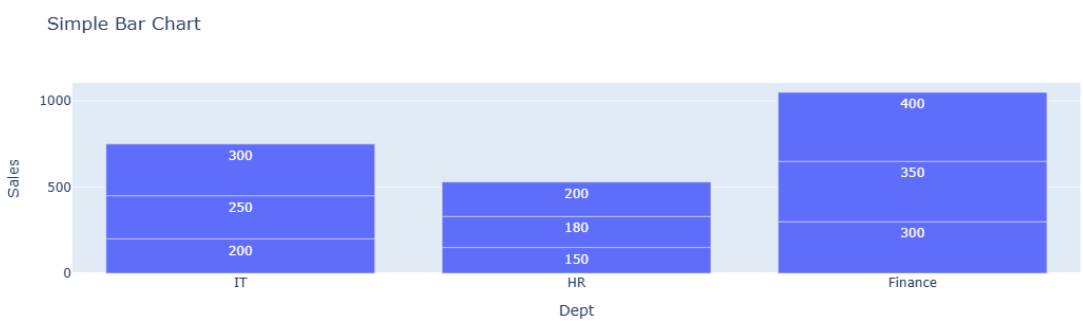
Relative → profit/loss type data

In [31]:
```
import plotly.express as px
import pandas as pd

# Bigger dataset
df = pd.DataFrame({
    'Dept': ['IT','IT','IT','HR','HR','HR','Finance','Finance','Finance'],
    'Year': ['2022','2023','2024']*3,
    'Sales': [200, 250, 300, 150, 180, 200, 300, 350, 400]
})

print(df)
```

```
      Dept  Year  Sales
0       IT  2022    200
1       IT  2023    250
2       IT  2024    300
3       HR  2022    150
4       HR  2023    180
5       HR  2024    200
6  Finance  2022    300
7  Finance  2023    350
8  Finance  2024    400
```

1. Simple Bar Chart

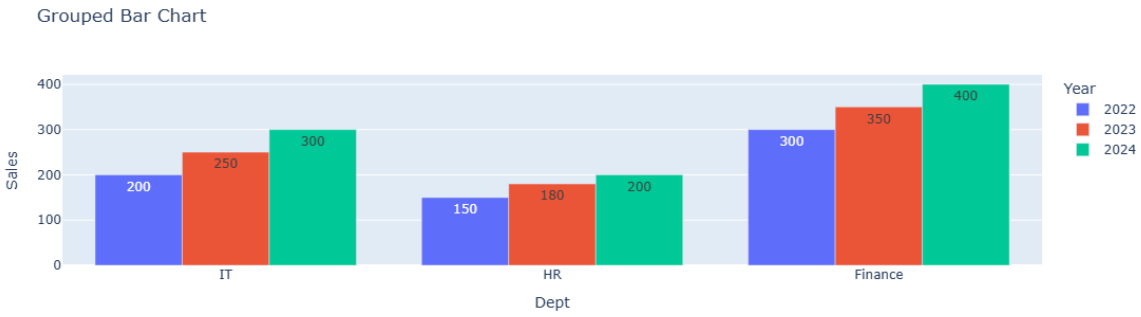
In [32]:
```
fig = px.bar(df, x='Dept', y='Sales',
             title="Simple Bar Chart",
             text_auto=True)
fig.show()
```



2. Grouped Bar Chart (barmode='group')

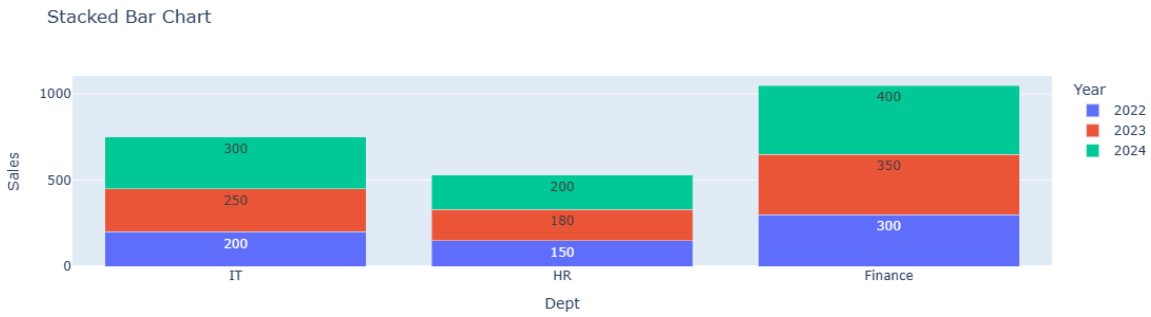
Vishal Acharya

In [33]:
```
fig = px.bar(df, x='Dept', y='Sales',
             color='Year',
             barmode='group',
             title="Grouped Bar Chart",
             text_auto=True)
fig.show()
```



3. Stacked Bar Chart (barmode='stack')

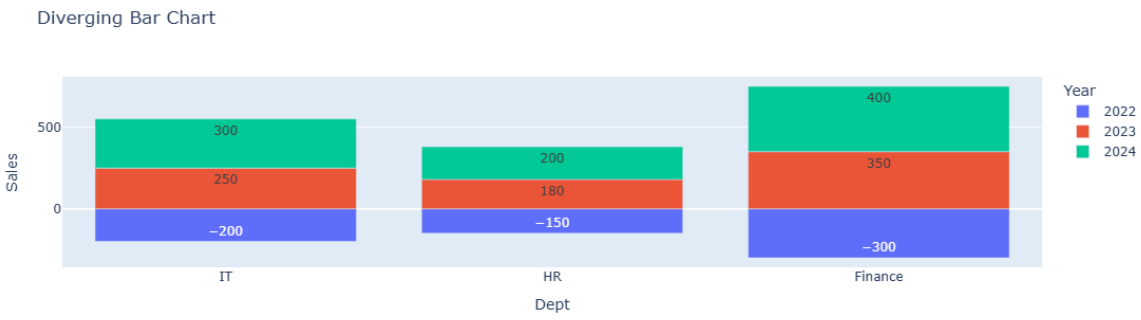
In [34]:
```
fig = px.bar(df, x='Dept', y='Sales',
             color='Year',
             barmode='stack',
             title="Stacked Bar Chart",
             text_auto=True)
fig.show()
```



4. Diverging Bar Chart (barmode='relative')

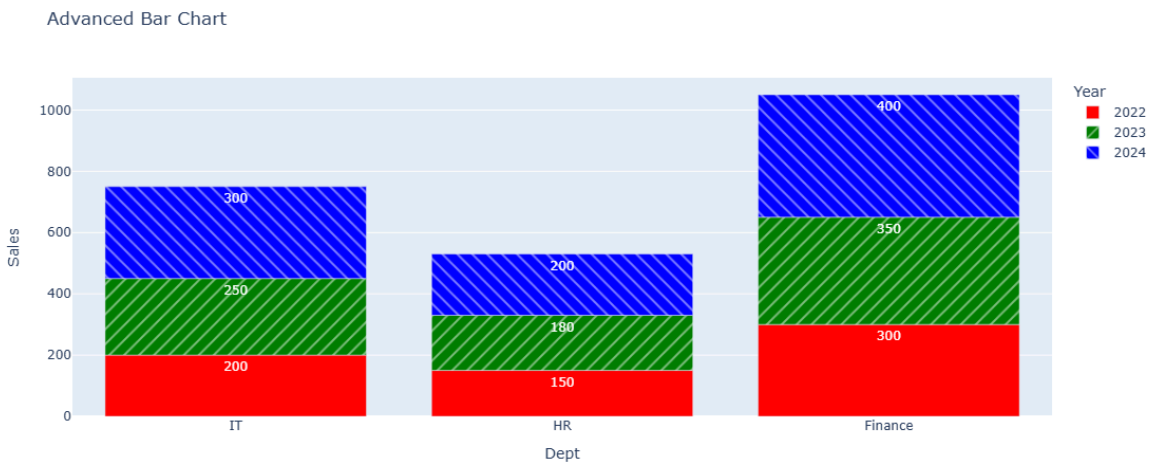
In [35]:
```
# Add negative values for diverging effect
df2 = df.copy()
df2.loc[df2['Year']=='2022', 'Sales'] *= -1

fig = px.bar(df2, x='Dept', y='Sales',
             color='Year',
             barmode='relative',
             title="Diverging Bar Chart",
             text_auto=True)
fig.show()
```

Vishal Acharya



Using More Parameters

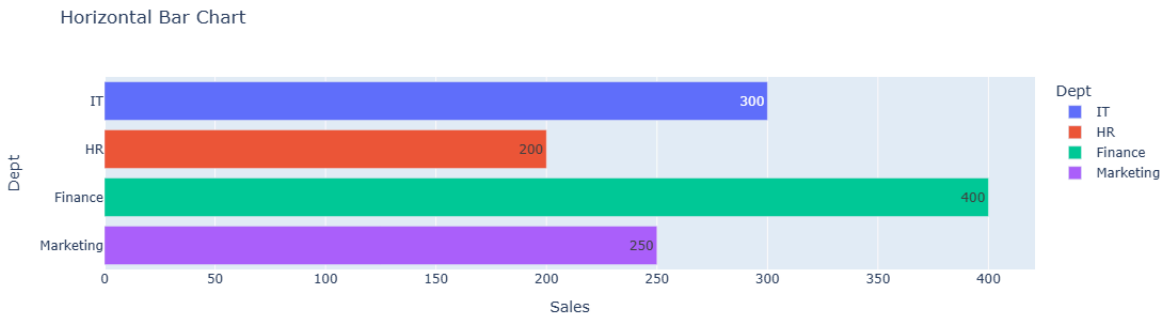
In [36]:
```
fig = px.bar(
    df,
    x='Dept',
    y='Sales',
    color='Year',
    width=800,
    height=500,
    title="Advanced Bar Chart",
    hover_data=['Year'],
    pattern_shape='Year',
    text_auto=True,
    orientation='v',
    color_discrete_sequence=['red','green','blue']
)
fig.show()
```



In [38]:
```
import plotly.express as px
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'Dept': ['IT', 'HR', 'Finance', 'Marketing'],
    'Sales': [300, 200, 400, 250]
})

# Horizontal bar chart
fig = px.bar(
    df,
    x='Sales',          # numeric → x-axis
```

Vishal Acharya

```
    y='Dept',           # category → y-axis
    orientation='h',    # horizontal
    title="Horizontal Bar Chart",
    text_auto=True,
    color='Dept'
)

fig.show()
```



px.scatter() → Creates 2D/3D scatter plots to show relationships between variables

x / y → Axes values

width / height → Figure size

title → Chart title

color → Grouping (different colors)

size → Bubble size

color_discrete_map → Custom color mapping

color_discrete_sequence → List of colors

symbol → Marker shape

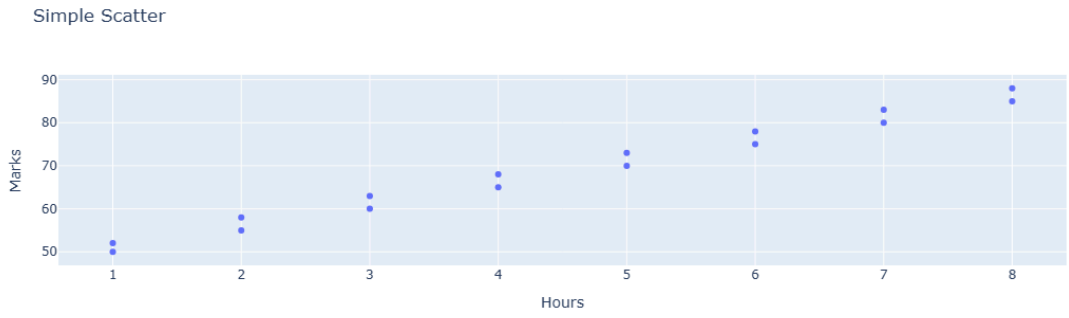
facet_col / facet_row → Split plots into subplots

In [39]:
```
import plotly.express as px
import pandas as pd

df = pd.DataFrame({
    'Hours': [1,2,3,4,5,6,7,8]*2,
    'Marks': [50,55,60,65,70,75,80,85,
              52,58,63,68,73,78,83,88],
    'Dept': ['IT','HR','Finance','IT','HR','Finance','IT','HR']*2,
    'Gender': ['M','F']*8,
    'Age': [18,19,20,21,22,23,24,25]*2
})
```

Vishal Acharya

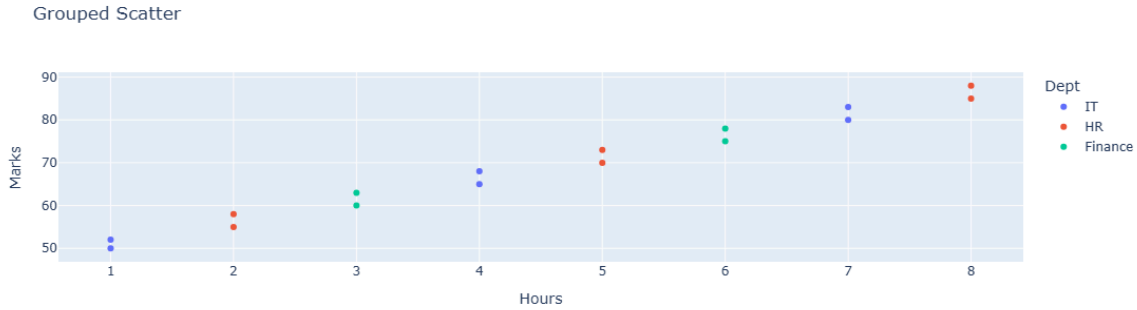
1. Simple 2D Scatter

In [40]:
```
px.scatter(df, x='Hours', y='Marks',
           title="Simple Scatter").show()
```



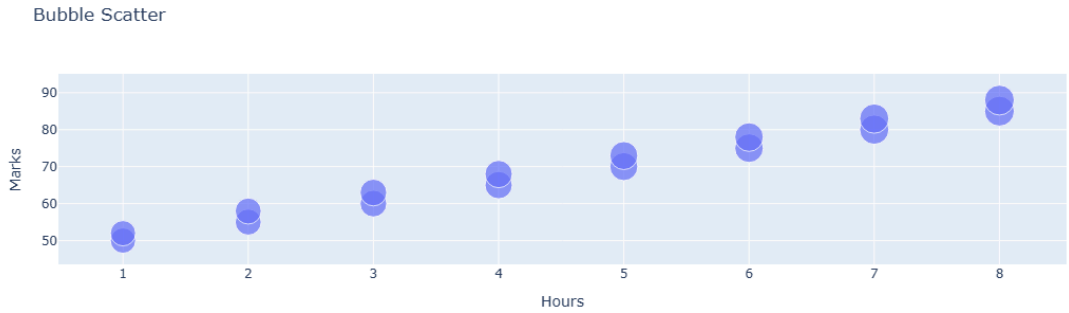
2. Group Scatter (color)

In [41]:
```
px.scatter(df, x='Hours', y='Marks',
           color='Dept',
           title="Grouped Scatter").show()
```



3. Bubble Scatter (size)

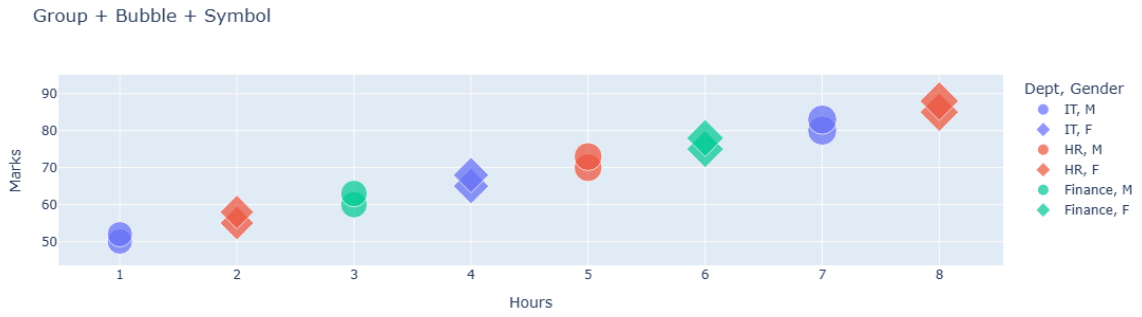
In [42]:
```
px.scatter(df, x='Hours', y='Marks',
           size='Age',
           title="Bubble Scatter").show()
```



4. Group + Bubble + Symbol

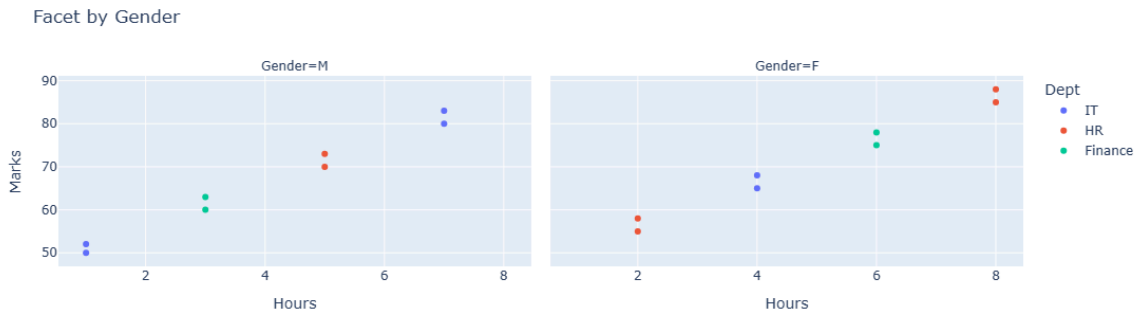
Vishal Acharya

In [43]:
```
px.scatter(df, x='Hours', y='Marks',
           color='Dept',
           size='Age',
           symbol='Gender',
           title="Group + Bubble + Symbol").show()
```



5. Facet (Multiple Plots)

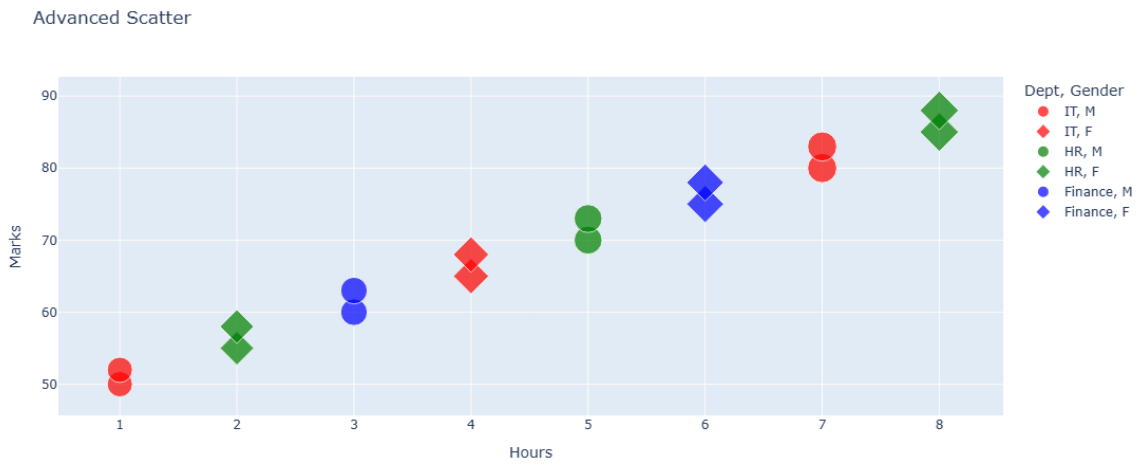
In [45]:
```
px.scatter(df, x='Hours', y='Marks',
           color='Dept',
           facet_col='Gender',#facet_row-"gender"
           title="Facet by Gender").show()
```



6. all parameter Scatter

In [46]:
```
px.scatter(
    df,
    x='Hours',
    y='Marks',
    color='Dept',
    size='Age',
    symbol='Gender',
    width=800,
    height=500,
    title="Advanced Scatter",
    color_discrete_sequence=['red','green','blue']
).show()
```

Vishal Acharya



Vishal Acharya

color → grouping

size → bubble effect

symbol → shape difference

facet → multiple subplots

3D → adds third dimension

scatter_matrix → pairwise relationships

px.scatter_3d(df, x, y, z, color, size, title):px.scatter_3d() → Creates a 3D scatter plot to visualize relationship between three variables

df → DataFrame

x, y, z → Axes values (3 dimensions)

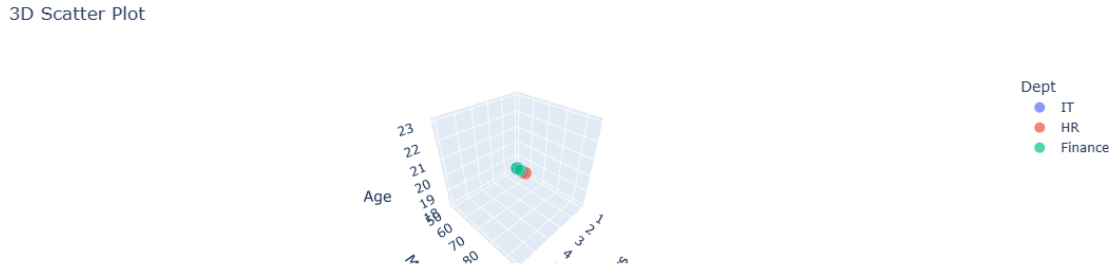
color → Grouping (different colors)

size → Bubble size

title → Chart title

In [47]:
```
import plotly.express as px
import pandas as pd

df = pd.DataFrame({
    'Hours': [1,2,3,4,5,6],
    'Marks': [50,60,70,80,90,95],
    'Age': [18,19,20,21,22,23],
    'Dept': ['IT','HR','Finance','IT','HR','Finance']
})

px.scatter_3d(
    df,
    x='Hours',
    y='Marks',
```

```
    z='Age',
    color='Dept',
    size='Age',
    title="3D Scatter Plot"
).show()
```



px.scatter_matrix() → Creates pairwise scatter plots for multiple columns

df → DataFrame

dimensions → List of columns to compare

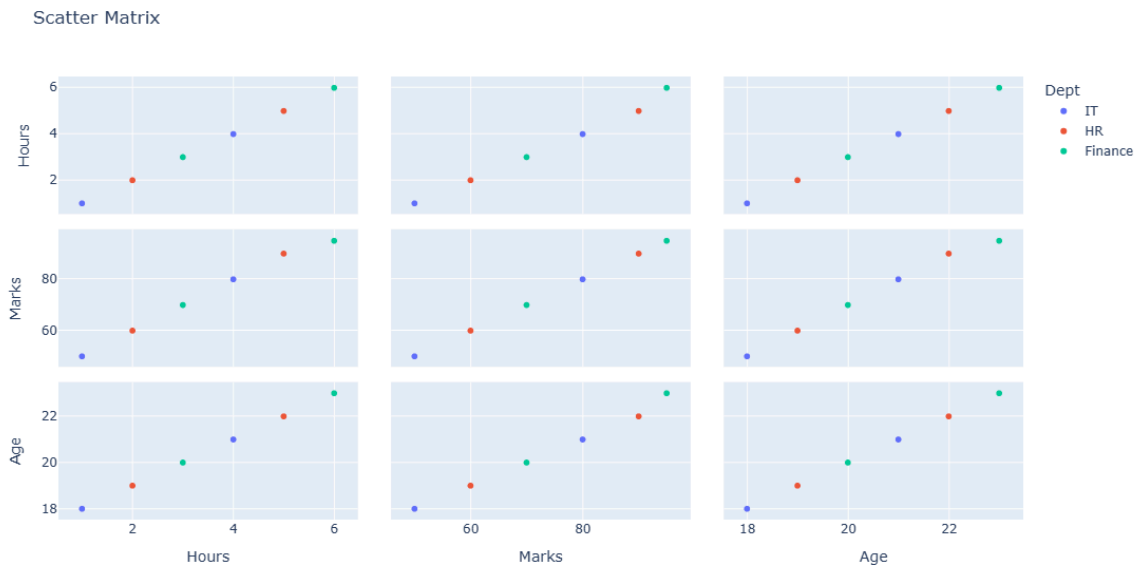
title → Chart title

color → Grouping

height / width → Size of chart

In [48]:
```
px.scatter_matrix(
    df,
    dimensions=['Hours', 'Marks', 'Age'],
    color='Dept',
    height=600,
    width=600,
    title="Scatter Matrix"
).show()
```

Vishal Acharya



px.line(df, x, y, color, markers, hover_data, color_discrete_map,title, color_discrete_sequence)

Single line → one category trend

Multi line → compare multiple categories

color → creates multiple lines

markers=True → shows points

hover_data → extra info

df → DataFrame

x / y → Axes values

color → Creates multiple lines (grouping)

markers → Shows points on line if True

hover_data → Extra info on hover

color_discrete_map → Custom color mapping

color_discrete_sequence → List of colors

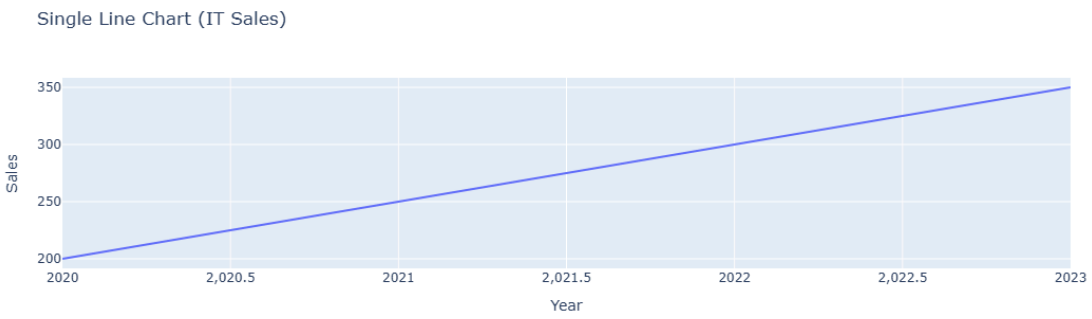
title → Chart title

Vishal Acharya

In [50]:
```
import plotly.express as px
import pandas as pd

df = pd.DataFrame({
    'Year': [2020,2021,2022,2023]*3,
    'Dept': ['IT']*4 + ['HR']*4 + ['Finance']*4,
    'Sales': [200,250,300,350, 150,180,200,220, 300,320,350,400]
})
```

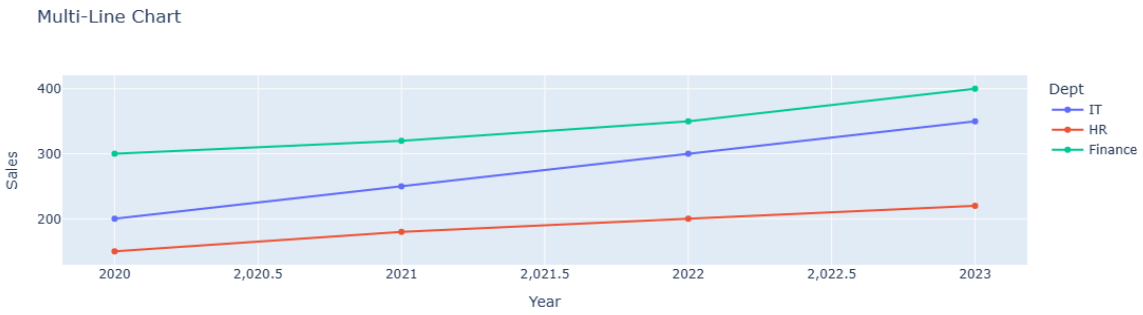
1. Single Line Chart

In [51]:
```
# Filter only IT for single line
df_it = df[df['Dept'] == 'IT']

px.line(
    df_it,
    x='Year',
    y='Sales',
    title="Single Line Chart (IT Sales)"
).show()
```



2. Multiple Line Chart

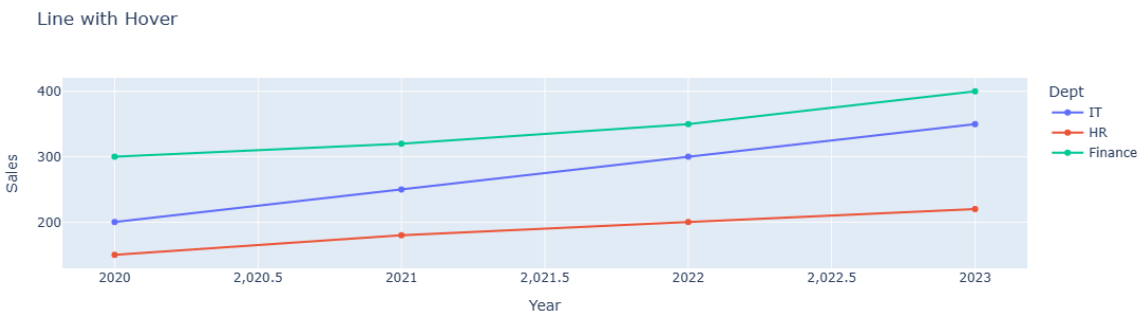
In [53]:
```
px.line(
    df,
    x='Year',
    y='Sales',
    color='Dept',          # multiple lines
    markers=True,
    title="Multi-Line Chart"
).show()
```



Vishal Acharya

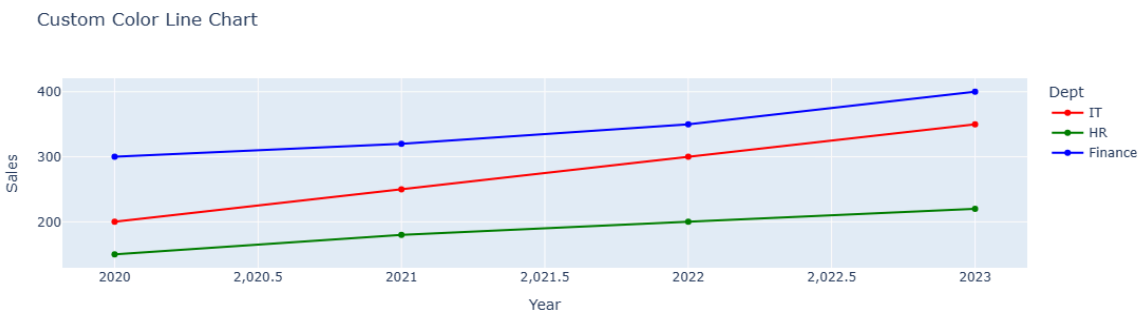
With Hover Data

In [54]:
```
px.line(
    df,
    x='Year',
    y='Sales',
    color='Dept',
    hover_data=['Dept'],
    markers=True,
    title="Line with Hover"
).show()
```



4. Custom Colors

In [55]:
```
px.line(
    df,
    x='Year',
    y='Sales',
    color='Dept',
    color_discrete_map={
        'IT': 'red',
        'HR': 'green',
        'Finance': 'blue'
    },
    markers=True,
    title="Custom Color Line Chart"
).show()
```

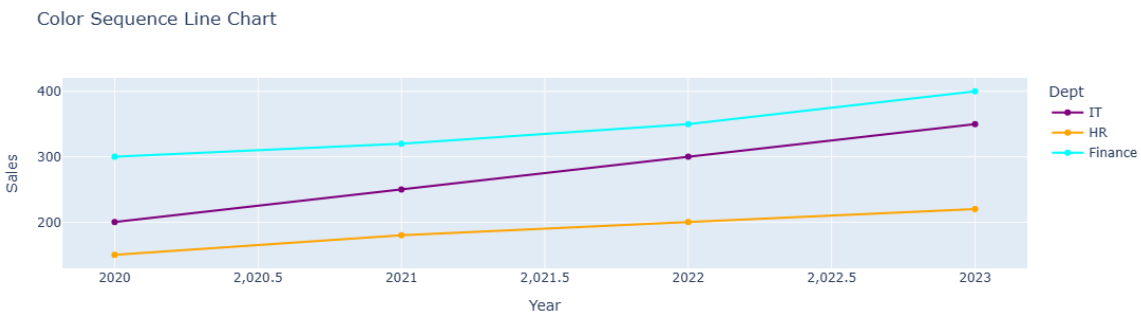


5. Using Color Sequence

In [56]:
```
px.line(
    df,
```

Vishal Acharya

```
    x='Year',
    y='Sales',
    color='Dept',
    color_discrete_sequence=['purple','orange','cyan'],
    markers=True,
    title="Color Sequence Line Chart"
).show()
```



px.histogram() → Creates a histogram to show frequency distribution of data

df → DataFrame

x → Column for distribution (main axis)

y → Optional (used with aggregation like sum/avg)

nbins → Number of bins (bars)

color → Grouping by category

histfunc → Aggregation ('count', 'sum', 'avg', 'min', 'max')

pattern_shape → Pattern styles for bars

text_auto → Show values on bars

color_discrete_map → Custom color mapping

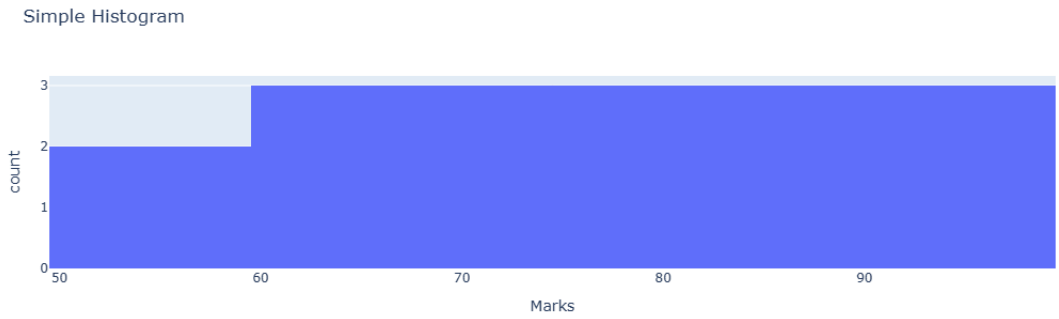
color_discrete_sequence → List of color

In [59]:
```
import plotly.express as px
import pandas as pd

df = pd.DataFrame({
    'Marks': [50,55,60,65,70,75,80,85,90,95,60,70,80,90],
    'Dept': ['IT','HR','Finance','IT','HR','Finance','IT','HR','Finance','IT','H
    'Age': [18,19,20,21,22,23,24,25,18,19,20,21,22,23]
})
```

1. Simple Histogram

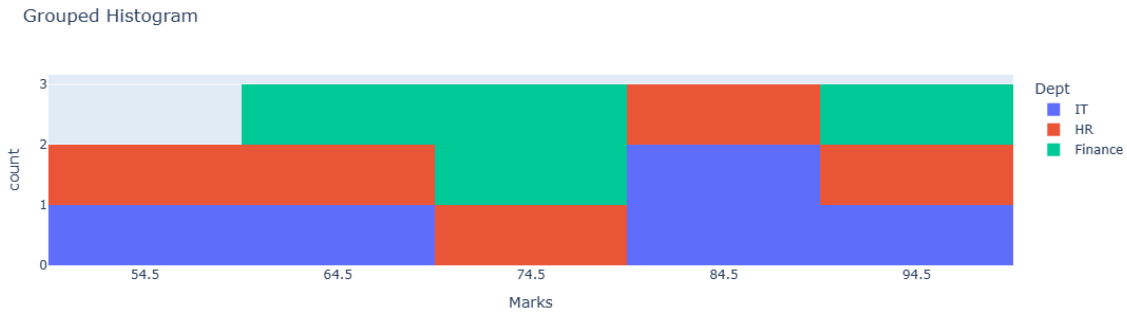
Vishal Acharya

In [60]:
```
px.histogram(
    df,
    x='Marks',
    nbins=5,
    title="Simple Histogram"
).show()
```



2. Histogram with Color (Grouped)

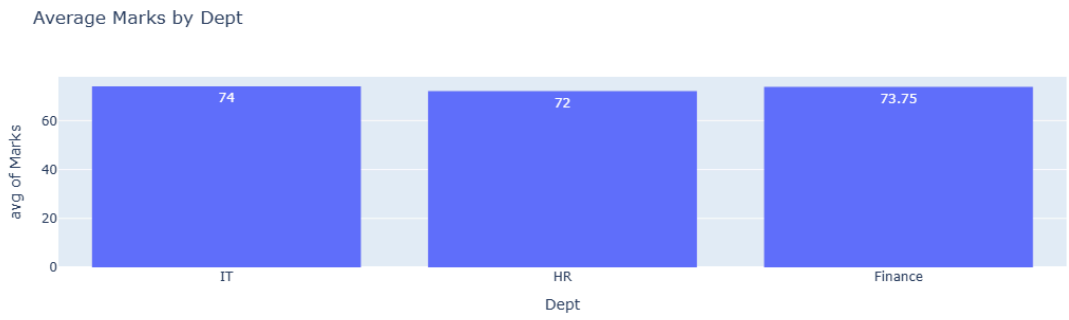
In [61]:
```
px.histogram(
    df,
    x='Marks',
    color='Dept',
    nbins=5,
    title="Grouped Histogram"
).show()
```



3. Histogram with histfunc (Aggregation)

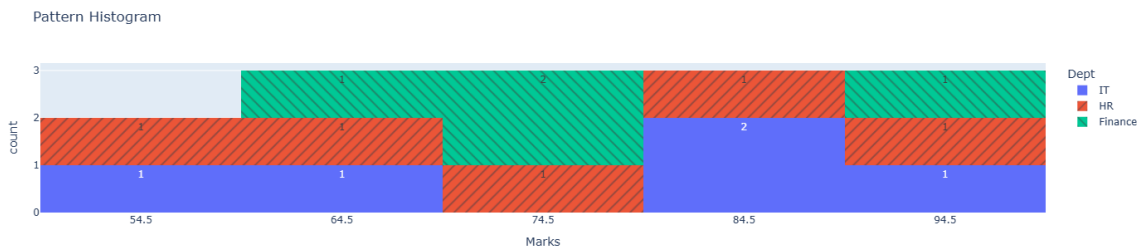
In [62]:
```
px.histogram(
    df,
    x='Dept',
    y='Marks',
    histfunc='avg',
    title="Average Marks by Dept",
    text_auto=True
).show()
```

Vishal Acharya



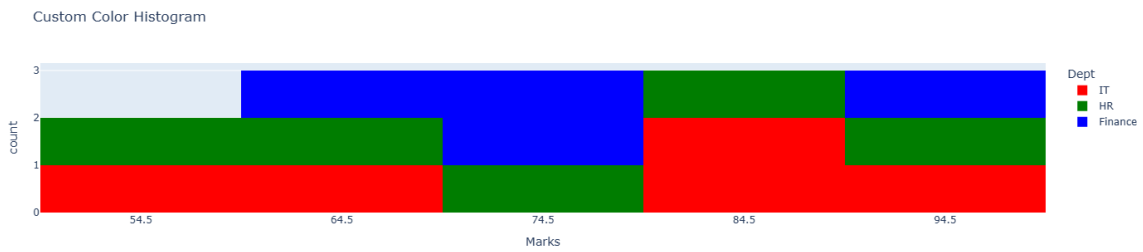
4. Pattern + Text

In [63]:
```
px.histogram(
    df,
    x='Marks',
    color='Dept',
    pattern_shape='Dept',
    text_auto=True,
    title="Pattern Histogram"
).show()
```



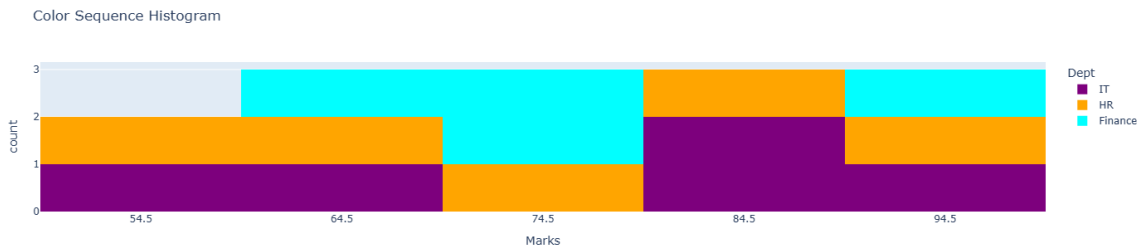
5. Custom Colors

In [64]:
```
px.histogram(
    df,
    x='Marks',
    color='Dept',
    color_discrete_map={
        'IT': 'red',
        'HR': 'green',
        'Finance': 'blue'
    },
    title="Custom Color Histogram"
).show()
```



Vishal Acharya

In [65]:
```
px.histogram(
    df,
    x='Marks',
    color='Dept',
    color_discrete_sequence=['purple','orange','cyan'],
    title="Color Sequence Histogram"
).show()
```



px.pie() → Creates a pie chart showing proportion/percentage of each category

values → Numeric values (used for size of slices)

names → Category labels

color_discrete_sequence → Colors of slices

title → Chart title

textinfo="percent+label" → Shows both % and name

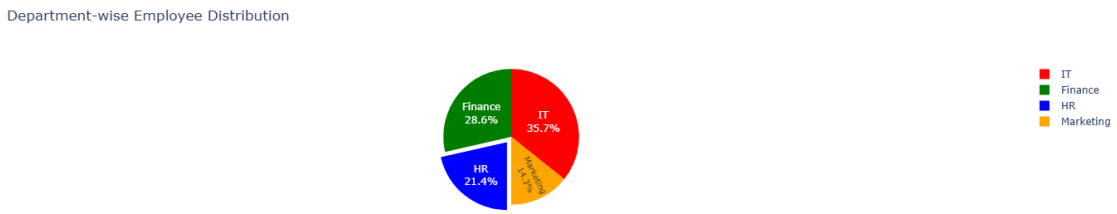
textposition → Position of text (inside / outside)

pull → Pulls out slice (highlight effect)

In [66]:
```
import plotly.express as px
import pandas as pd

# Create Data
data = pd.DataFrame({
    'Dept': ['IT', 'HR', 'Finance', 'Marketing'],
    'Employees': [50, 30, 40, 20]
})

# Create Pie Chart
fig = px.pie(
    data,
    values='Employees',                  # values to calculate percentage
    names='Dept',                        # labels
    color_discrete_sequence=['red','green','blue','orange'],
    title="Department-wise Employee Distribution"
)

# Update chart
fig.update_traces(
    textinfo="percent+label",   # show percentage + label
```

Vishal Acharya

```
    textposition="inside",      # text inside slices
    pull=[0, 0.1, 0, 0]         # highlight HR slice
)

fig.show()
```



Vishal Acharya

pd.get_dummies(df, columns, drop_first=True) → Converts categorical columns into dummy/indicator variables (False & True)

df → Input DataFrame

columns → Column(s) to encode

drop_first=True → Drops first category to avoid dummy variable trap

drop_first=True → removes one column

In [68]:
```
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'Name': ['A', 'B', 'C', 'D'],
    'Dept': ['IT', 'HR', 'Finance', 'IT']
})

print("Original Data:\n", df, "\n")

# One-hot encoding
df_encoded = pd.get_dummies(df, columns=['Dept'], drop_first=True)

print("After get_dummies:\n", df_encoded)
```

```
Original Data:
   Name     Dept
0    A       IT
1    B       HR
2    C  Finance
3    D       IT

After get_dummies:
   Name  Dept_HR  Dept_IT
0    A    False     True
1    B     True    False
2    C    False    False
3    D    False     True
```

np.where(condition, x, y) → Returns x if condition is True, else y

In [69]:
```python
import numpy as np
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'Marks': [45, 60, 75, 85, 30]
})

# Apply multiple conditions
df['Grade'] = np.where(
    (df['Marks'] >= 80), 'A',                        # condition 1
    np.where(
        (df['Marks'] >= 50) & (df['Marks'] < 80), 'B',    # condition 2
        'C'                                           # else
    )
)

print(df)
```

```
   Marks Grade
0     45     C
1     60     B
2     75     B
3     85     A
4     30     C
```

In [70]:
```python
import pandas as pd

#  ◆  Series with different index
s1 = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
s2 = pd.Series([1, 2, 3], index=['a', 'b', 'd'])

print("Series s1:\n", s1)
print("Series s2:\n", s2, "\n")

# Addition (default → NaN)
print("s1 + s2:\n", s1 + s2, "\n")

# Addition with fill_value
print("s1.add(s2, fill_value=0):\n", s1.add(s2, fill_value=0), "\n")
```

Vishal Acharya

```python
# 🔹 DataFrame with different columns
df1 = pd.DataFrame({
    'A': [10, 20],
    'B': [30, 40]
})

df2 = pd.DataFrame({
    'B': [5, 6],
    'C': [7, 8]
})

print("DF1:\n", df1)
print("DF2:\n", df2, "\n")

# Addition (default → NaN)
print("df1 + df2:\n", df1 + df2, "\n")

# Addition with fill_value
print("df1.add(df2, fill_value=0):\n", df1.add(df2, fill_value=0), "\n")


# 🔹 Combined operation
result = df1.add(df2, fill_value=0) * 2
print("Combined Result (add + multiply):\n", result)
```

Vishal Acharya

```
Series s1:
 a    10
b    20
c    30
dtype: int64
Series s2:
 a    1
b    2
d    3
dtype: int64

s1 + s2:
 a    11.0
b    22.0
c     NaN
d     NaN
dtype: float64

s1.add(s2, fill_value=0):
 a    11.0
b    22.0
c    30.0
d     3.0
dtype: float64

DF1:
     A   B
0  10  30
1  20  40
DF2:
    B  C
0  5  7
1  6  8

df1 + df2:
     A   B   C
0 NaN  35 NaN
1 NaN  46 NaN

df1.add(df2, fill_value=0):
       A   B    C
0  10.0  35  7.0
1  20.0  46  8.0

Combined Result (add + multiply):
       A   B     C
0  20.0  70  14.0
1  40.0  92  16.0
```

train_test_split() → Splits dataset into training and testing (validation) sets

X → Input features (independent variables)

y → Target variable (dependent variable)

test_size → Proportion of data for testing (e.g., 0.2 = 20%)

Vishal Acharya

train_size → Proportion for training (optional)

random_state → Ensures same random split every time

In [71]:
```
from sklearn.model_selection import train_test_split
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'Hours': [1,2,3,4,5,6,7,8],
    'Marks': [50,55,60,65,70,75,80,85],
    'Result': ['Fail','Fail','Pass','Pass','Pass','Pass','Pass','Pass']
})

# Features (X) and Target (y)
X = df[['Hours', 'Marks']]
y = df['Result']

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,     # 25% test data
    random_state=42,    # fixed split

)

print("X_train:\n", X_train, "\n")
print("X_test:\n", X_test, "\n")
print("y_train:\n", y_train, "\n")
print("y_test:\n", y_test)
```

```
X_train:
    Hours  Marks
0      1     50
7      8     85
2      3     60
4      5     70
3      4     65
6      7     80

X_test:
    Hours  Marks
1      2     55
5      6     75

y_train:
 0    Fail
7    Pass
2    Pass
4    Pass
3    Pass
6    Pass
Name: Result, dtype: object

y_test:
 1    Fail
5    Pass
Name: Result, dtype: object
```

Vishal Acharya

1. Cross-Validation (Intro)

One-line Explanation

Cross-validation → Splits data into multiple parts to evaluate model performance more reliably

Idea (K-Fold)

Data is divided into K parts (folds)

Train on K-1 folds, test on 1 fold

Repeat K times

In [72]:
```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LinearRegression
import pandas as pd

# Data
df = pd.DataFrame({
    'X': [1,2,3,4,5,6,7,8],
    'y': [2,4,6,8,10,12,14,16]
})

X = df[['X']]
y = df['y']

model = LinearRegression()

# 5-fold cross-validation
scores = cross_val_score(model, X, y, cv=5)

print("Scores:", scores)
print("Average Score:", scores.mean())
```

```
Scores: [ 1.  1.  1. nan nan]
Average Score: nan
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1288:
UndefinedMetricWarning: R^2 score is not well-defined with less than two samples.
  warnings.warn(msg, UndefinedMetricWarning)
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1288:
UndefinedMetricWarning: R^2 score is not well-defined with less than two samples.
  warnings.warn(msg, UndefinedMetricWarning)
```

In [ ]:

Vishal Acharya